



**Pontificia Universidad
Católica del Ecuador**
Seréis mis testigos

MANABÍ

**PONTIFICIA UNIVERSIDAD CATÓLICA DEL ECUADOR
SEDE MANABÍ**

CARRERA DE SOFTWARE

TRABAJO DE TITULACIÓN

**DESARROLLO DE UNA PLATAFORMA SAAS QUE INTEGRA
MODULOS CON IA PARA LA GESTIÓN INTEGRAL DE PYMES
GASTRONÓMICAS EN PORTOVIEJO**

LÍNEA DE INVESTIGACIÓN

**INGENIERÍA DE SOFTWARE Y SISTEMAS COMPUTACIONALES PARA LA
TRANSFORMACIÓN DIGITAL EMPRESARIAL**

SUBLÍNEA DE INVESTIGACIÓN

**TRANSFORMACIÓN DIGITAL DE PYMES Y SISTEMAS SAAS
PROCESOS EMPRESARIALES**

**PREVIO AL TÍTULO DE
INGENIERO EN SOFTWARE**

AUTORES

**LUIGGI ANDREE ARTEAGA SANTANA
CHRISTIAN ORLANDO SARAGURO ENCALADA**

TUTOR

JOSE NARANJO, M.ENG.

PORTOVIEJO, FEBRERO 2026

Certificación del Tutor

En mi calidad de tutor del trabajo de integración curricular, certifico haber revisado el presente manuscrito de investigación, el mismo que se ajusta a las normas vigentes de la Pontificia Universidad Católica del Ecuador Sede Manabí, cumpliendo la Normativa del Trabajo de Integración Curricular; en consecuencia, es apto para su presentación y sustentación. En la ciudad de Portoviejo a los quince días del mes de febrero de dos mil veintiséis.

JOSE NARANJO, M.ENG.

CI: 1003528674

Tutor(a)

Acta de aprobación del Tribunal

El jurado examinador aprueba el presente trabajo de integración curricular en nombre de la Pontificia Universidad Católica del Ecuador, Sede Manabí.

En la ciudad de Portoviejo a los quince días del mes de febrero de dos mil veintiséis.

Nombre del revisor 1

CI:0000000000

Lector Revisor1(a)

Nombre del revisor 2

CI:0000000000

Lector Revisor2(a)

JOSE NARANJO, M.ENG

CI: 1003528674

Tutor(a)

Declaración de Originalidad

Este manuscrito no contiene ningún tipo de material que ha sido aceptado para la obtención de un título universitario en otra institución, excepto en forma de información de soporte que ha sido debidamente citada en mi trabajo. Este trabajo es de total responsabilidad del autor, quien declara bajo juramento que ninguna sección de este trabajo de integración curricular infringe los derechos de autor de nadie.

En la ciudad de Portoviejo a los quince días del mes de febrero de dos mil veintiséis.

LUIGGI ARTEAGA

C.I.: 1314693597

Autor

CHRISTIAN SARAGURO

C.I.: 1350274484

Autor

Declaración de Derecho del Autor

Autorizo a la Pontificia Universidad Católica del Ecuador a distribuir este manuscrito de investigación en medios físicos y electrónicos con el fin de promover la divulgación de mis resultados a la comunidad científica y a la sociedad en general. Adicionalmente autorizo el uso de los contenidos de esta investigación como bibliografía para fines académicos, por cualquier medio o procedimiento, citando como fuente de información al autor de este trabajo. En la ciudad de Portoviejo a los quince días del mes de febrero de dos mil veintiséis.

LUIGGI ARTEAGA

C.I.: 1314693597

Autor

CHRISTIAN SARAGURO

C.I.: 1350274484

Autor

Dedicatoria

Dedico este trabajo, en primer lugar, a Dios, por ser mi fortaleza en los momentos difíciles y por permitirme alcanzar una de las metas más importantes de mi vida.

A mi familia, por su amor incondicional, sacrificio, paciencia y por creer siempre en mí, incluso cuando yo mismo dudaba. Su apoyo constante ha sido el motor que me impulsó a seguir adelante.

A mis padres, por ser ejemplo de esfuerzo, responsabilidad y perseverancia, y por enseñarme que los sueños se alcanzan con trabajo y dedicación.

Finalmente, dedico este logro a todas aquellas personas que con una palabra de aliento, un consejo o un gesto de apoyo hicieron posible la culminación de esta etapa tan significativa en mi vida.

Agradecimientos

Agradezco profundamente a Dios por brindarme la fortaleza, salud y sabiduría necesarias para culminar esta importante etapa de mi formación profesional.

Mi sincero agradecimiento a mi familia, quienes han sido mi mayor apoyo y motivación constante, brindándome su amor, comprensión y respaldo incondicional en cada paso de este proceso.

Agradezco de manera especial a mis docentes, quienes con su conocimiento, paciencia y orientación contribuyeron significativamente a mi desarrollo académico y profesional, permitiéndome adquirir las herramientas necesarias para la realización de este trabajo.

Un agradecimiento particular a mi tutor/a de tesis, por su guía, tiempo y valiosos aportes durante el desarrollo de esta investigación, siendo un pilar fundamental para su culminación exitosa.

Finalmente, agradezco a todas las personas que, de manera directa o indirecta, colaboraron en la realización de este proyecto y formaron parte de este importante logro personal y profesional.

Resumen

Este trabajo presenta el diseño, desarrollo y validación de una plataforma SaaS integral para la digitalización de PYMES gastronómicas en Portoviejo. El objetivo principal es optimizar los procesos operativos de gestión de pedidos, inventarios, delivery y atención al cliente mediante la implementación de módulos tecnológicos integrados y accesibles.

La propuesta consiste en una arquitectura multiplataforma que integra un sistema de gestión de pedidos con geolocalización en tiempo real, módulo de control de inventarios automatizado, panel administrativo con analítica de indicadores clave de desempeño (KPIs), y un chatbot basado en inteligencia artificial utilizando tecnología RAG (Retrieval-Augmented Generation) para atención automatizada. El desarrollo se realizó bajo metodología ágil Scrum, empleando tecnologías web modernas y bases de datos relacionales escalables.

Para evaluar la solución, se aplicó un enfoque metodológico mixto que combina encuestas estructuradas, entrevistas semiestructuradas y pruebas de usabilidad (métricas SUS) con usuarios reales de 2 PYMES seleccionadas por conveniencia. Los resultados esperados incluyen la demostración de mejoras en tiempos de respuesta operativa, reducción de errores en gestión de pedidos e inventarios, disminución de costos asociados a plataformas de terceros, y validación positiva de usabilidad y experiencia de usuario. Esta investigación se alinea con la Agenda de Transformación Digital del Ecuador 2022–2025 y contribuye al fortalecimiento competitivo del sector gastronómico de Portoviejo, reconocido por UNESCO como Ciudad Creativa de la Gastronomía.

Palabras clave: PYMES, gastronomía, digitalización, SaaS, Portoviejo.

Abstract

This work presents the design, development, and validation of an integrated SaaS platform for the digitalization of gastronomic SMEs in Portoviejo. The main objective is to optimize operational processes for order management, inventory, delivery, and customer service through the implementation of integrated and accessible technological modules.

The proposal consists of a multiplatform architecture that integrates an order management system with real-time geolocation, automated inventory control module, administrative dashboard with key performance indicators (KPIs) analytics, and an AI-based chatbot using RAG (Retrieval-Augmented Generation) technology for automated customer service. The development was carried out under the Scrum agile methodology, employing modern web technologies and scalable relational databases.

To evaluate the solution, a mixed methodological approach was applied, combining structured surveys, semi-structured interviews, and usability testing (SUS metrics) with real users from 2 SMEs selected by convenience. Expected results include demonstrating improvements in operational response times, reduction of errors in order and inventory management, decreased costs associated with third-party platforms, and positive validation of usability and user experience. This research aligns with Ecuador's Digital Transformation Agenda 2022–2025 and contributes to the competitive strengthening of Portoviejo's gastronomic sector, recognized by UNESCO as a Creative City of Gastronomy.

Keywords: SMEs, gastronomy, digitalization, SaaS, Portoviejo.

Tabla de Contenido

Introducción	1
Antecedentes	2
Contexto del sector gastronómico en Portoviejo	2
Composición empresarial y digitalización en Ecuador	2
Barreras para la digitalización de PYMES	3
Políticas nacionales de transformación digital	3
Justificación de una solución SaaS para PYMES gastronómicas	4
Definición del problema	4
Preguntas de investigación	7
Pregunta general	7
Preguntas específicas	7
Objetivos	8
Objetivo general	8
Objetivos específicos	8
Alcance	9
Metodología de la Investigación	10
Tipo y diseño de investigación	10
Población y muestra	10

Técnicas de recolección de datos	11
Instrumentos	11
Análisis de datos	11
Método	12
Enfoque y tipo de investigación	12
Diseño de la investigación	12
Población y muestra	13
Población	13
Muestra	13
Criterios de inclusión y exclusión	13
Técnicas e instrumentos de recolección de datos	14
Técnicas aplicadas	14
Instrumentos	14
Diseño de instrumentos	14
Encuestas	14
Entrevistas	15
Pruebas de usabilidad	15
Piloto y SUS	15
Gestión y organización de datos	15
Procedimiento	15
FASE 1: Planificación	16

FASE 2: Desarrollo	18
Diseño y desarrollo del frontend (React Native + Expo)	18
Diseño y desarrollo del panel web (Next.js + Tailwind CSS)	20
Diseño y desarrollo del backend (NestJS, capas, módulos)	21
Diseño de la base de datos (modelo relacional, multinegocio)	26
Roles, permisos y seguridad (JWT, RBAC)	29
Flujo funcional (proceso de pedido de punta a punta)	34
Estado actual del desarrollo	36
Resumen del procedimiento	38
Características pendientes y futuras	38
FASE 3: Implementación, despliegue y pruebas	40
FASE 4: Resultados	41
Resultados en dos locales	41
Usabilidad (SUS) y adopción	42
Análisis comparativo y observaciones	43
Limitaciones	43
Mejoras porcentuales por indicador	43
Disponibilidad y latencia observada	44
Análisis de datos	44
Consideraciones éticas	45
Resultados	47

Resultados esperados	47
Costo estimado y financiamiento	48
Cronograma de actividades	48
Presentación de resultados	50
Discusión	51
Digitalización actual	51
Diseño y desarrollo multiplataforma	51
Gestión de repartidores y delivery	52
Panel administrativo y analítica	52
Chatbot con inteligencia artificial (RAG)	52
Usabilidad y aceptación del sistema	53
Síntesis	53
Conclusiones	55
Conclusiones principales	55
Importancia de la transformación digital	55
Viabilidad de la plataforma SaaS	55
Recomendaciones y Trabajo Futuro	56
Alianzas y participación de actores	56
Referencias bibliográficas	58
Apéndice A	59

Lista de Figuras

1.	Diagrama de prototipo del frontend móvil	19
2.	Diagrama de prototipo del panel web administrativo	20
3.	Arquitectura modular del backend (NestJS)	24
4.	Arquitectura de despliegue y componentes	25
5.	Diagrama entidad-relación (ER) del esquema de datos	26
6.	Flujo de autenticación y autorización	30
7.	Diagrama de secuencia del proceso de pedido	36
8.	<i>Cronograma de actividades del proyecto de titulación</i>	<i>49</i>
9.	<i>Diagrama entidad-relación de la base de datos de la plataforma SaaS</i>	<i>60</i>
10.	<i>Diagrama de red y arquitectura de la plataforma SaaS web y móvil</i>	<i>60</i>
11.	<i>Diagrama de flujo del funcionamiento general de la plataforma SaaS</i>	<i>61</i>

Lista de Tablas

1.	<i>Indicadores operativos antes y después por local</i>	42
2.	<i>Puntajes de usabilidad SUS por local</i>	42
3.	<i>Adopción por rol al cierre de validación</i>	42
4.	<i>Variación porcentual por indicador (Local A y B)</i>	44
5.	<i>Métricas de rendimiento y latencia</i>	44
6.	<i>Disponibilidad por servicio</i>	44
7.	<i>Resultados esperados de la investigación</i>	47
8.	<i>Costo estimado del proyecto</i>	48

Introducción

En las últimas décadas, la digitalización empresarial ha transformado radicalmente los modelos operativos y competitivos a nivel mundial. La adopción masiva de tecnologías digitales ha permitido a las organizaciones optimizar procesos, reducir costos operativos, mejorar la experiencia del cliente y tomar decisiones basadas en datos en tiempo real. Sectores como el comercio electrónico, la logística, los servicios financieros y la manufactura han experimentado cambios estructurales profundos mediante la implementación de sistemas ERP, plataformas de gestión integrada, automatización de procesos e inteligencia artificial aplicada. Sin embargo, esta transformación digital no ha sido uniforme ni equitativa: mientras que grandes corporaciones disponen de recursos técnicos, económicos y humanos para adoptar soluciones tecnológicas avanzadas, las Pequeñas y Medianas Empresas (PYMES) continúan enfrentando barreras significativas de acceso, costo, capacitación y adaptabilidad tecnológica.

A nivel latinoamericano y particularmente en Ecuador, las PYMES constituyen la columna vertebral del tejido empresarial, representando más del 90 % de las unidades productivas registradas y generando una proporción significativa del empleo nacional. No obstante, diversos estudios e informes institucionales evidencian que la mayoría de estas empresas mantiene niveles de digitalización limitados, especialmente en sectores tradicionales como la gastronomía, donde los procesos operativos continúan dependiendo de gestión manual, canales de comunicación no integrados y ausencia de herramientas automatizadas para la toma de decisiones. Esta brecha digital no solo limita su competitividad frente a empresas tecnificadas o plataformas digitales consolidadas, sino que también afecta su capacidad de adaptación, escalabilidad y sostenibilidad en mercados cada vez más exigentes, donde los consumidores demandan inmediatez, conveniencia, transparencia y experiencias omnicanal.

Frente a este panorama, las soluciones SaaS (Software as a Service) representan una oportunidad estratégica para democratizar el acceso a tecnología empresarial avanzada sin requerir inversiones iniciales elevadas, infraestructura propia o conocimientos técnicos especializados. Bajo esta perspectiva, el presente estudio se centra en evaluar la factibilidad, diseño, desarrollo y validación de una plataforma SaaS integral orientada específicamente a las PYMES gastronómicas de Portoviejo, una ciudad reconocida internacionalmente por la UNESCO como Ciudad Creativa de la Gastronomía, pero cuyos negocios gastronómicos locales aún enfrentan desafíos significativos en su proceso de modernización digital.

Antecedentes

Contexto del sector gastronómico en Portoviejo

Portoviejo fue declarada Ciudad Creativa de la Gastronomía por la UNESCO en 2019, un reconocimiento internacional que posiciona a la gastronomía local como un elemento estratégico tanto cultural como económico. Este distintivo subraya el potencial del sector para impulsar el desarrollo turístico, generar empleo y fortalecer la identidad regional. Las PYMES gastronómicas constituyen el núcleo de este sector, desempeñando un rol fundamental en la preservación de tradiciones culinarias y en la dinamización de la economía local.

Composición empresarial y digitalización en Ecuador

A nivel nacional, el tejido empresarial ecuatoriano está compuesto en gran parte por PYMES. Según datos del Observatorio de la PyME, cerca del 90,5 % de las empresas registradas en Ecuador son microempresas, seguidas por pequeñas y medianas. En el caso de la provincia de Manabí, donde se encuentra Portoviejo, las PYMES también juegan un rol importante en la generación de empleo y el desarrollo local.

Barreras para la digitalización de PYMES

Uno de los desafíos principales para estas empresas es la limitada adopción de tecnologías digitales. Si bien existe un reconocimiento creciente de su importancia, muchas PYMES siguen enfrentando barreras estructurales como costos elevados, falta de conocimientos técnicos, ausencia de capacitación especializada y sistemas tecnológicos poco adaptados a sus necesidades y capacidades operativas.

En muchos negocios gastronómicos pequeños, la gestión operativa (por ejemplo, inventarios y pedidos) continúa siendo manual o con herramientas básicas. Esta falta de automatización puede generar errores, desperdicios y dificultades para proyectar la demanda, lo que impacta negativamente en la rentabilidad y eficiencia. Además, el servicio al cliente todavía depende de canales tradicionales como llamadas telefónicas o WhatsApp, limitando la capacidad de ofrecer una experiencia profesional, rápida y escalable.

Por otra parte, la comisión que cobran algunas plataformas de delivery externas puede representar un costo significativo. Aunque no existen estudios exactos para todas las PYMES gastronómicas de Portoviejo, análisis de mercado muestran que estas comisiones pueden ser elevadas, reduciendo los márgenes de pequeños emprendimientos y afectando su sostenibilidad económica.

Políticas nacionales de transformación digital

La Agenda de Transformación Digital del Ecuador 2022–2025 establece una hoja de ruta institucional para impulsar la adopción de tecnologías en distintos sectores, incluyendo las PYMES. Entre sus objetivos está promover infraestructura tecnológica, facilitar la capacitación y crear un entorno favorable para que las empresas pequeñas accedan a soluciones digitales.

Justificación de una solución SaaS para PYMES gastronómicas

Frente a estas problemáticas, una solución SaaS (Software como Servicio) adaptada a PYMES gastronómicas locales emerge como una alternativa viable y estratégica. Un sistema SaaS puede ofrecer módulos integrados para la gestión de pedidos, control de inventarios, analítica empresarial y atención al cliente automatizada (mediante chatbots), sin requerir inversiones grandes en infraestructura o conocimientos técnicos avanzados por parte de los usuarios.

Por lo tanto, desarrollar una plataforma SaaS que responda a las necesidades operativas, técnicas y económicas de las PYMES gastronómicas de Portoviejo constituye una estrategia viable y alineada con las metas institucionales nacionales de digitalización y con las aspiraciones locales de modernización, competitividad y crecimiento sostenible del sector.

Definición del problema

En el sector gastronómico de la ciudad de Portoviejo, las Pequeñas y Medianas Empresas (PYMES) desempeñan un rol fundamental en la economía local, aportando significativamente a la generación de empleo, al dinamismo comercial y a la identidad gastronómica de la región. A pesar de su importancia, este sector enfrenta un proceso de digitalización lento e incompleto, caracterizado por la utilización de métodos tradicionales en la gestión operativa, administrativa y comercial. En un entorno donde la tecnología se ha convertido en un componente esencial para la competitividad y la sostenibilidad, muchas de estas empresas continúan utilizando procesos manuales para administrar pedidos, inventarios y servicios de delivery, dificultando su adaptación a las nuevas exigencias del mercado digital.

El problema principal identificado es la falta de soluciones tecnológicas accesibles, integradas y adaptadas al contexto local, que permitan a las PYMES gastronómicas gestionar de forma eficiente sus operaciones diarias. Actualmente, la mayoría de estos negocios no dispone de herramientas centralizadas para controlar inventarios, administrar pedidos, gestionar repartidores o brindar

atención al cliente mediante sistemas automatizados. Esto genera desorganización, tiempos de respuesta elevados, errores en la toma de pedidos y pérdidas económicas que afectan directamente la calidad del servicio y la rentabilidad del negocio. La ausencia de una plataforma tecnológica integral limita la capacidad de estas empresas para modernizarse y competir frente a cadenas más grandes o plataformas de delivery externas.

Diversos estudios y diagnósticos sobre transformación digital en Ecuador evidencian que las PYMES mantienen un nivel de adopción tecnológica limitado, especialmente en actividades tradicionales como la gastronomía. Aunque representan más del 90 % del tejido empresarial del país, informes como la Agenda de Transformación Digital del Ecuador 2022–2025 señalan que estas empresas aún enfrentan barreras significativas para modernizar sus procesos, debido a limitaciones económicas, escasa capacitación técnica y la falta de soluciones accesibles y adaptadas a su escala operativa. En Portoviejo, pese a ser reconocida por la UNESCO como Ciudad Creativa de la Gastronomía en 2019, la mayoría de los negocios gastronómicos continúa operando sin sistemas integrados de gestión para pedidos, inventarios y atención al cliente, lo cual genera procesos manuales, inconsistencias y baja eficiencia operativa. Esta evidencia muestra una brecha marcada entre el potencial gastronómico de la ciudad y el nivel real de digitalización presente en sus pequeñas y medianas empresas, lo que confirma la existencia de un problema estructural que afecta su competitividad y sostenibilidad.

Las causas del problema se relacionan principalmente con:

1. El alto costo de las soluciones tecnológicas disponibles en el mercado, que suelen estar orientadas a empresas medianas o grandes y no se ajustan al presupuesto de las PYMES locales.
2. La falta de capacitación tecnológica del personal, lo cual dificulta la adopción de plataformas complejas.
3. La ausencia de una oferta local de software adaptable, que incluya en una sola plataforma

módulos de delivery, inventarios, atención automatizada y analítica.

4. La dependencia de plataformas de terceros, que cobran comisiones elevadas y no permiten gestionar los procesos de manera personalizada.

Estas causas se combinan para generar un rezago tecnológico que limita el crecimiento y la modernización del sector gastronómico.

Si el problema no se soluciona, las PYMES gastronómicas de Portoviejo continuarán enfrentando dificultades operativas que derivan en pérdidas económicas, baja productividad y disminución de la calidad del servicio. La falta de digitalización provocará que estos negocios sigan siendo menos competitivos frente a empresas que ya adoptaron herramientas tecnológicas avanzadas. Además, se incrementarán los errores en la toma de pedidos, el desperdicio de insumos por falta de control de inventarios y los tiempos de entrega elevados debido a una mala gestión de repartidores. A largo plazo, esta situación limitará su capacidad para crecer, atraer nuevos clientes y mantenerse en un mercado que demanda rapidez, precisión y experiencia digital.

Resolver este problema es fundamental porque permitirá impulsar la transformación digital del sector gastronómico local, mejorar la competitividad de las PYMES y promover la adopción de tecnologías accesibles e inteligentes. El desarrollo de una plataforma SaaS integrada contribuirá a optimizar los procesos operativos, reducir costos, mejorar la toma de decisiones mediante analítica de datos y elevar la calidad del servicio al cliente mediante sistemas automatizados. Además, esta solución se alinea con los objetivos nacionales de digitalización y con la visión de Portoviejo como una ciudad innovadora y creativa. Abordar esta problemática beneficia no solo a los negocios, sino también al ecosistema económico local al fomentar la modernización, sostenibilidad y crecimiento del sector gastronómico.

Preguntas de investigación

Pregunta general

¿De qué manera el desarrollo e implementación de una plataforma SaaS integral puede contribuir a la digitalización, optimización operativa y mejora de la atención al cliente en las PYMES gastronómicas de la ciudad de Portoviejo?

Preguntas específicas

1. ¿Cuáles son las principales barreras tecnológicas, económicas y operativas que limitan la digitalización de las PYMES gastronómicas de Portoviejo?
2. ¿Qué módulos, características y funcionalidades son esenciales para mejorar la gestión de delivery, inventarios y procesos administrativos en el contexto gastronómico local?
3. ¿Cómo influye la integración de un módulo de inteligencia artificial basado en tecnología RAG en la eficiencia, rapidez y calidad de la atención al cliente?
4. ¿En qué medida la adopción de la plataforma SaaS desarrollada puede reducir costos operativos, optimizar tiempos de respuesta y mejorar la toma de decisiones estratégicas en las PYMES usuarias?
5. ¿Cuál es la percepción de usabilidad y satisfacción del usuario final respecto a la plataforma propuesta?

Objetivos

Objetivo general

Desarrollar y validar una plataforma SaaS que integre módulos de gestión operativa e inteligencia artificial, para optimizar los procesos de pedidos, inventarios, delivery y atención al cliente de las PYMES gastronómicas de Portoviejo.

Objetivos específicos

1. Realizar un diagnóstico del nivel de digitalización y un levantamiento detallado de requisitos funcionales y no funcionales con las PYMES objetivo.
2. Diseñar la arquitectura técnica, de software y de despliegue de la plataforma, definiendo su estructura modular y los protocolos de seguridad e integración.
3. Construir la plataforma mediante el desarrollo e integración simultánea de sus componentes modulares principales.
4. Implementar los módulos de gestión de pedidos (con geolocalización en tiempo real), control de inventarios, administración de repartidores y optimización de rutas, y el dashboard analítico con KPIs.
5. Integrar un chatbot basado en el modelo RAG dentro de la plataforma, para automatizar la atención al cliente y el soporte operativo.
6. Evaluar la plataforma integral mediante pruebas de usabilidad (SUS), rendimiento, funcionalidad y aceptación con usuarios reales en un entorno piloto, cuantificando su impacto en la eficiencia operativa.

Alcance

El presente proyecto se enmarca en el diseño, desarrollo y validación de una plataforma SaaS integral dirigida a la digitalización de las Pequeñas y Medianas Empresas (PYMES) del sector gastronómico de Portoviejo. Con el fin de establecer límites claros y garantizar una ejecución realista, se define el alcance del proyecto en dos dimensiones: los elementos que serán desarrollados e implementados, y aquellos que quedan excluidos de esta fase.

El proyecto abarca la creación completa de una solución tecnológica operativa, desde su concepción hasta su validación en un entorno controlado. En primer lugar, se realizará un análisis de requisitos basado en un diagnóstico del nivel de digitalización actual de las PYMES objetivo, mediante técnicas como encuestas y entrevistas, para derivar especificaciones funcionales y no funcionales. El diseño resultante contempla una arquitectura de software modular, compuesta por un backend desarrollado con NestJS que expone una API RESTful, una aplicación móvil multiplataforma construida con React Native para clientes y repartidores, y un panel administrativo web implementado con Next.js. La persistencia de datos se gestionará mediante una base de datos relacional PostgreSQL.

La plataforma integrará módulos funcionales centrales para la gestión operativa: un sistema de pedidos con registro de geolocalización de direcciones, un módulo de control de inventarios básico, un panel de administración de repartidores con procesos de aprobación y asignación manual de pedidos, y un dashboard analítico que presente indicadores clave de desempeño (KPIs) derivados de los datos transaccionales. Como componente innovador, se desarrollará e integrará un chatbot basado en el modelo de Generación Aumentada por Recuperación (RAG) para automatizar respuestas a consultas frecuentes de clientes.

La validación de la solución se llevará a cabo mediante un piloto controlado en al menos dos PYMES gastronómicas, donde se evaluará la usabilidad del sistema a través de la escala SUS (System Usability Scale), se verificará la funcionalidad de los flujos críticos y se medirá el impacto en métricas operativas como el tiempo de registro de pedidos y la tasa de errores. El despliegue

para esta validación se realizará en infraestructura en la nube, utilizando un servidor VPS para el backend y la base de datos, Vercel para el panel web, y se distribuirá la aplicación móvil como archivo APK para dispositivos Android.

Para garantizar la viabilidad y el enfoque del proyecto, se excluyen funcionalidades avanzadas y consideraciones de escala industrial. Esto incluye: el desarrollo de una arquitectura multi-tenant; la integración con pasarelas de pago; la optimización automática de rutas y el live-tracking de repartidores; las notificaciones en tiempo real (WebSockets/push); y módulos complementarios como marketing, reseñas o gestión de proveedores. Tampoco se contempla el desarrollo de una aplicación nativa para iOS, el despliegue de infraestructura de alta disponibilidad, una validación a escala sectorial, o un plan de soporte y mantenimiento post-implementación.

Metodología de la Investigación

La investigación se desarrolla bajo un enfoque mixto (cuantitativo y cualitativo), debido a que combina análisis estadísticos, recolección de datos estructurados y evaluación de percepciones de usuarios.

Tipo y diseño de investigación

1. Investigación aplicada
2. Diseño no experimental
3. Estudio descriptivo y exploratorio
4. Desarrollo iterativo bajo metodología Scrum

Población y muestra

1. Población: PYMES gastronómicas de Portoviejo.

2. Muestra: 2 PYMES seleccionadas por conveniencia, disponibilidad y características operativas representativas.

Técnicas de recolección de datos

1. Encuestas estructuradas para identificar niveles de digitalización.
2. Entrevistas semiestructuradas para comprender necesidades operativas.
3. Pruebas de usabilidad (UX) del prototipo.
4. Observación directa de procesos internos.

Instrumentos

1. Formularios digitales (Google Forms o similares).
2. Guías de entrevista aprobadas por el tutor.
3. Herramientas de grabación y toma de tiempos.
4. Métricas de usabilidad (SUS, métricas de eficiencia y eficacia).

Análisis de datos

1. Análisis estadístico descriptivo para datos cuantitativos.
2. Análisis categórico y temático para información cualitativa.
3. Evaluación comparativa pre y post implementación del prototipo.

Método

Enfoque y tipo de investigación

La investigación se desarrolló bajo un enfoque metodológico mixto (cualitativo-cuantitativo), con predominancia del componente aplicativo-desarrollativo. Se adoptó este enfoque para obtener una comprensión integral del problema: el componente cualitativo permitió profundizar en las percepciones, necesidades y experiencias de los usuarios, mientras que el cuantitativo posibilitó la medición objetiva de variables operativas y de usabilidad.

El estudio se clasifica como investigación aplicada de tipo tecnológica, dado que su propósito central es el diseño, construcción y validación de un artefacto de software (la plataforma SaaS) para resolver una problemática concreta en un contexto real. El diseño es no experimental, descriptivo y transversal, ya que se observaron y describieron las variables en su contexto natural sin manipulación deliberada, en un período específico.

Diseño de la investigación

El presente trabajo sigue la metodología Investigación Basada en Diseño (IBD), elegida por ser una metodología de investigación poderosa para desarrollar y evaluar artefactos (productos, procesos, métodos, modelos, etc.) que resuelven problemas del mundo real.

No existe un único modelo canónico de fases, ya que distintos autores han propuesto esquemas similares pero con variaciones. Sin embargo, todas comparten un núcleo cíclico e iterativo de construcción y evaluación.

Se presenta el siguiente esquema integrando los modelos clave de Hevner et al. (2004) y Peffers

et al. (2007).

Población y muestra

Población

La población estuvo conformada por Pequeñas y Medianas Empresas del sector gastronómico de la ciudad de Portoviejo, caracterizadas por operar con recursos limitados y por distintos niveles de adopción tecnológica en sus procesos administrativos y operativos.

Muestra

La muestra se integró por dos PYMES seleccionadas mediante muestreo no probabilístico por conveniencia.

Criterios de inclusión y exclusión

Los criterios de inclusión fueron: disponibilidad de los propietarios, representatividad de sus procesos y disposición a evaluar una solución para la gestión de pedidos, inventarios y atención al cliente.

Esta selección aportó información relevante y contextualizada para el diseño y validación del prototipo.

Para la caracterización de la muestra se recogieron datos básicos del negocio (tamaño, número de empleados, volumen de pedidos, herramientas tecnológicas utilizadas) que permitieron contextualizar los hallazgos.

Se definieron criterios de exclusión (por ejemplo, empresas sin operaciones regulares durante el periodo de estudio o con procesos imposibles de observar) y se especificó el procedimiento de acercamiento y consentimiento informado.

Aunque el tamaño de la muestra es acotado, se priorizó la profundidad del análisis y la pertinencia de los casos seleccionados para validar el prototipo en escenarios reales.

Técnicas e instrumentos de recolección de datos

Técnicas aplicadas

Se emplearon técnicas complementarias para recabar información cuantitativa y cualitativa: encuestas estructuradas, entrevistas semiestructuradas, pruebas de usabilidad y observación directa de los procesos internos de las PYMES participantes.

Instrumentos

Los instrumentos incluyeron formularios digitales (Google Forms) para encuestas, guías de entrevista revisadas y aprobadas por el tutor, y herramientas de registro y cronometraje durante las pruebas de usabilidad. Se aplicaron métricas estándar de usabilidad, como el System Usability Scale (SUS), junto con indicadores de eficiencia y eficacia para evaluar el desempeño del prototipo.

Diseño de instrumentos

El diseño de instrumentos siguió buenas prácticas de claridad y neutralidad en la formulación de ítems.

Encuestas

Las encuestas incluyeron preguntas cerradas (escala Likert) y abiertas para capturar tanto opiniones cuantificables como comentarios cualitativos.

Entrevistas

Las guías de entrevista se estructuraron por temática (experiencia actual, necesidades, percepción del prototipo) y se aplicaron en sesiones semiestructuradas.

Pruebas de usabilidad

En pruebas de usabilidad se definieron tareas representativas (realizar un pedido, consultar estado de entrega, crear producto en el panel admin) con escenarios controlados y tiempos registrados; se identificaron errores, puntos de fricción y necesidades de mejora.

Piloto y SUS

Previo a la aplicación, se realizó una verificación piloto de comprensión de ítems para detectar ambigüedades y ajustar redacción.

En el caso del SUS, se siguió el procedimiento estándar de puntuación y normalización, interpretando resultados en rangos de aceptabilidad.

Gestión y organización de datos

Los datos fueron centralizados y organizados para facilitar su análisis, asegurando consistencia entre las diferentes fuentes.

Procedimiento

Esta sección describe, en orden, el diseño y desarrollo del frontend móvil, del panel web administrativo, del backend con NestJS y del modelo de base de datos relacional. Cada subsección inicia con su objetivo y alcances para facilitar la lectura y ubicar el rol que cumple dentro de la plataforma.

FASE 1: Planificación

Esta fase establece las decisiones estratégicas y el marco técnico del proyecto, asegurando coherencia entre objetivos, alcance y capacidades tecnológicas. Se definió el MVP y se seleccionaron tecnologías que maximizan mantenibilidad, velocidad de desarrollo y calidad de la experiencia de usuario, alineadas con buenas prácticas actuales.

Se adoptó una arquitectura por capas y dominios, con separación de responsabilidades entre cliente móvil, cliente web y servicios de negocio. La plataforma se estructuró en tres repositorios principales para favorecer independencia de ciclos de vida, pipelines de integración continua y control de versiones por componente:

Backend (NestJS): Un repositorio dedicado a la API y lógica de negocio, implementado con NestJS y TypeScript. NestJS aporta una arquitectura modular basada en controladores, servicios y repositorios, inyección de dependencias y tipado estático, lo que reduce errores y mejora testabilidad. Su integración natural con herramientas de seguridad (JWT, Guards) y validación de datos (DTOs con pipes) permite construir servicios robustos y auditables. La modularidad por dominios (Usuarios, Pedidos, Productos, Auth) facilita escalar y evolucionar sin afectar otras áreas, y se alinea con prácticas de diseño como DDD. La contenedorización con Docker y despliegue en la nube se integran de forma directa, y el enfoque en TypeScript fortalece la mantenibilidad del código a largo plazo 3.

Mobile (React Native + Expo): Un repositorio para la aplicación móvil multiplataforma. React Native habilita entregar una experiencia nativa en iOS y Android desde una única base de código, optimizando tiempos y costos. Expo simplifica todo el ciclo de vida (desarrollo con recarga en vivo, acceso a APIs nativas, compilación y distribución), disminuyendo barreras operativas y acelerando validaciones con usuarios. La UI se organiza con Tamagui, que provee componentes y sistema de estilos coherente y reutilizable entre móvil y web, facilitando consistencia visual y productividad. Esta elección permite iterar rápido sobre flujos críticos (registro, pedidos, entregas) y asegurar un rendimiento adecuado en dispositivos de gama media.

Web (Next.js + React + Tailwind CSS): Un repositorio para el panel administrativo web. Next.js ofrece una experiencia web nativa moderna, con enrutamiento por archivos, Server-Side Rendering y Static Site Generation cuando corresponde, optimización de recursos y soporte a componentes de servidor. Esto mejora tiempo de primer render, accesibilidad y percepción de desempeño en escritorio. React aporta el modelo declarativo y ecosistema ampliamente adoptado, y Tailwind CSS acelera el estilado con clases utilitarias consistentes, manteniendo diseño limpio y adaptable. Separar el panel en un repositorio propio permite evolucionar su UX/análítica con ciclos distintos al móvil, manteniendo foco en necesidades administrativas y reportes.

La separación en tres repositorios independientes aporta beneficios claros: aislamiento de fallos y cambios, despliegues desacoplados, ownership por equipo/componente, menor complejidad de revisión y permisos más precisos. Operativamente, cada repositorio puede tener su pipeline CI/CD, pruebas y ambientes de staging propios, reduciendo acoplamiento y facilitando releases controladas.

La base de datos seleccionada es relacional (PostgreSQL) por su idoneidad para el dominio: el sistema requiere integridad referencial entre entidades centrales (Usuarios, Productos, Categorías, Pedidos y sus Items, Negocios), transacciones atómicas para operaciones compuestas (creación de pedidos y detalle), y consultas analíticas con uniones para construir reportes (ventas por periodo, productos más vendidos, tiempos de entrega). Un RDBMS garantiza consistencia mediante claves foráneas y restricciones, soporta índices para consultas críticas y facilita agregaciones SQL para métricas. Frente a una base NoSQL, el modelo relacional brinda mayor seguridad en estados consistentes y en auditorías, y se alinea con la necesidad de reportabilidad sin sacrificar rendimiento. Para el soporte multinegocio, se adopta un discriminador por empresa en tablas principales, que permite segmentar datos por negocio manteniendo simplicidad en una sola instancia y dejando abierta la evolución a enfoques más aislados si la escala lo demanda 5.

Durante esta fase se elaboraron prototipos de navegación de la app móvil y del panel administrativo, con el objetivo de validar flujos, jerarquías de información y criterios de usabilidad con actores clave. Los prototipos, reflejados en 1 y 2, guiaron decisiones de estructura y estilos. Se

definieron criterios de éxito mensurables (reducción de tiempos y errores operativos, aceptabilidad de usabilidad mediante SUS), supuestos y riesgos (adopción móvil, latencia, escalabilidad inicial), y se planificó el trabajo en sprints según Scrum con backlog priorizado y metas por iteración, asegurando trazabilidad y foco en entregables de valor.

FASE 2: Desarrollo

Esta fase abarca el diseño y desarrollo de la plataforma multinegocio. A continuación se detallan los componentes construidos.

Diseño y desarrollo del frontend (React Native + Expo)

Objetivo del frontend: ofrecer una experiencia clara y eficiente para clientes y repartidores, con navegación predecible, validaciones tempranas y comunicación robusta con la API.

El frontend móvil se implementó con React Native y Expo, lo que permite desarrollar una aplicación nativa multiplataforma a partir de una sola base de código. React Native facilita la reutilización entre iOS y Android sin comprometer el rendimiento; Expo simplifica el ciclo de vida de la app al ofrecer herramientas integradas para compilar, desplegar y probar sin configurar proyectos nativos desde cero (campusMVP, s.f.).

Además, Expo expone funcionalidades nativas desde JavaScript (cámara, notificaciones, sensores) y habilita un flujo ágil con recarga en vivo y publicación mediante enlaces o códigos QR, favoreciendo la colaboración y las pruebas (campusMVP, s.f.).

La aplicación ofrece flujos diferenciados para dos roles: Cliente y Repartidor, además de pantallas de inicio de sesión y registro. La navegación se gestiona con la librería de React Native (p. ej., React Navigation), que permite transicionar entre vistas, mantener historiales y pasar parámetros.

Cada rol visualiza únicamente las opciones pertinentes; esta segregación mejora la experiencia y refuerza la seguridad, complementada con validaciones en el backend.

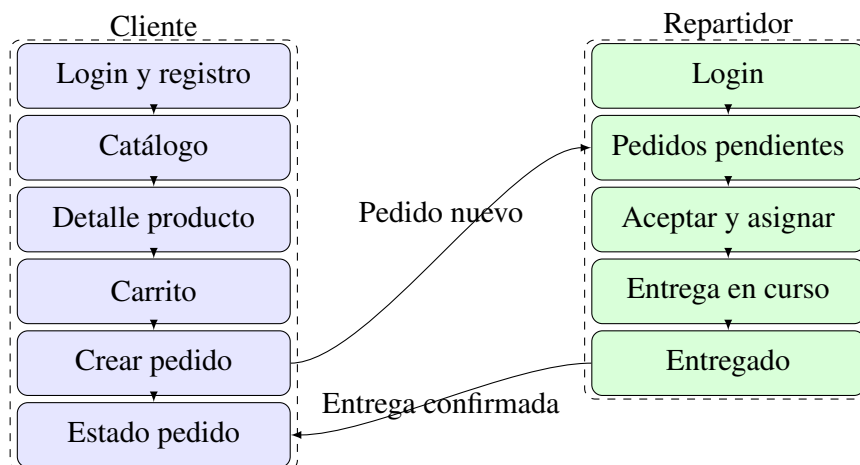


Figura 1. Diagrama de prototipo del frontend móvil

La arquitectura del frontend móvil privilegia la simplicidad y la previsibilidad del estado. Se emplea estado local y patrones de elevación de estado donde corresponde, con manejo de formularios controlados y validación básica en cliente (tipos de entrada, obligatoriedad, formatos).

La comunicación con la API se realiza mediante solicitudes HTTP que envían y reciben JSON; se contemplan casos de error (credenciales inválidas, falta de conectividad, respuestas 4xx/5xx) con mensajes claros al usuario y estrategias de reintento cuando es pertinente.

La interfaz se diseñó con principios de consistencia visual y accesibilidad, cuidando contrastes, tamaños de toque y retroalimentación tras acciones.

Para el diseño visual se empleó Tamagui, biblioteca de UI y sistema de estilos compatible con React Native y React web, que provee componentes y temas para construir interfaces coherentes y responsivas (Tamagui, s.f.).

En el proyecto se estilizaron botones, tarjetas y formularios con componentes Tamagui, logrando consistencia y mantenibilidad. Su enfoque multiplataforma facilita la reutilización futura de componentes en una versión web, así como la personalización temática por negocio.

Diseño y desarrollo del panel web (Next.js + Tailwind CSS)

Objetivo del panel web: proveer herramientas administrativas para gestionar catálogo, pedidos y repartidores, con formularios validados, vistas responsivas y flujos CRUD claros.

Se desarrolló además un panel web administrativo con React y Next.js. Aunque el panel es interno, se aprovechó el renderizado del lado del servidor (SSR) en vistas específicas de dashboard cuando resultó conveniente.

El estilado se realizó con Tailwind CSS, framework utility-first que permite componer diseños rápidos y consistentes mediante clases utilitarias (Tailwind CSS, s.f.). Su configuración admite alta personalización, útil para adaptar paletas o temas por cliente.

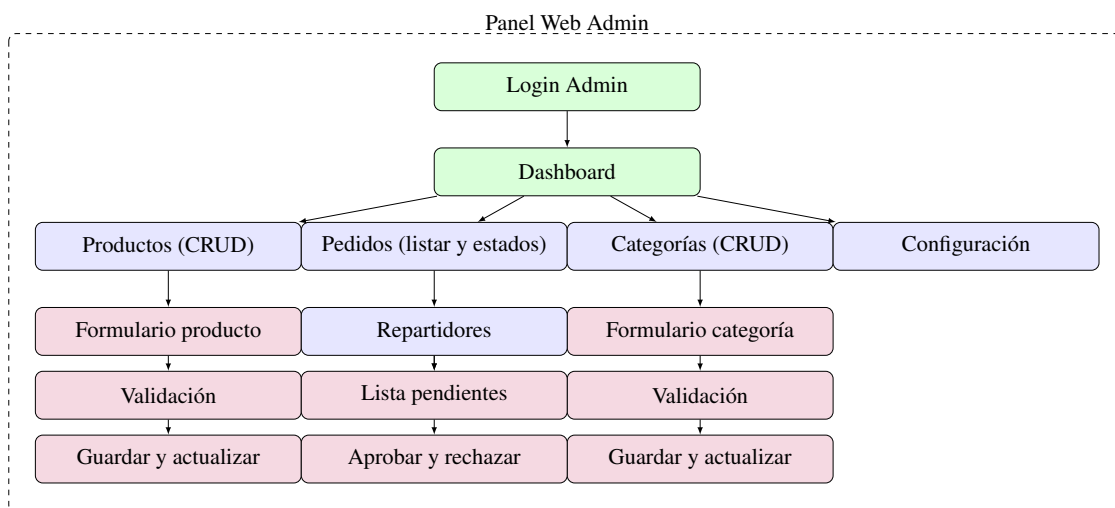


Figura 2. Diagrama de prototipo del panel web administrativo

En el panel se aplicaron formularios validados con reglas de negocio simples (campos requeridos, formatos, rangos) y se implementaron flujos habituales de CRUD con confirmaciones y mensajes de éxito/error.

La estructura de navegación organiza secciones por dominio (productos, categorías, pedidos, repartidores) y proporciona filtros básicos (por estado, fecha) para facilitar la gestión. Se cuidó la responsividad para operar en distintos tamaños de pantalla, y se diseñaron tablas y listas con paginación prevista para escenarios de datos voluminosos.

El panel permite crear y editar productos, gestionar categorías y aprobar o rechazar repartidores que solicitan unirse. Los formularios incorporan validaciones de campos; las vistas de lista presentan pedidos en curso y solicitudes pendientes.

Cuando un repartidor se registra desde la app móvil, su cuenta queda "pendiente" hasta la revisión administrativa, tras la cual puede ser habilitada o rechazada, garantizando control de calidad.

Ambos frontends consumen el mismo backend mediante API REST y JSON. La separación de responsabilidades mantiene el frontend enfocado en la experiencia y presentación, mientras la lógica de negocio y la persistencia residen en el backend.

En materia de rendimiento y experiencia, se optimizaron listas y vistas frecuentes y se cuidó la retroalimentación visual en operaciones que pueden tardar (carga de catálogo, envío de pedidos).

Se delinearon buenas prácticas para el manejo de imágenes (compresión, tamaños adecuados) y para el control de estados vacíos y errores de red, mejorando la robustez de la interfaz en condiciones reales.

Diseño y desarrollo del backend (NestJS, capas, módulos)

Objetivo del backend: exponer servicios estables y seguros, aplicar reglas de negocio y coordinar persistencia con una arquitectura modular mantenible.

El backend se implementó con NestJS, framework de Node.js de arquitectura modular que promueve buenas prácticas mediante el uso de TypeScript, decoradores e inyección de dependencias.

Frente a Express puro, NestJS aporta una estructura clara basada en módulos, controladores y servicios, facilitando la escalabilidad y el mantenimiento del código por dominios de negocio (Usuarios, Productos, Pedidos, Autenticación, entre otros).

Cada módulo agrupa tres componentes fundamentales: un controlador que define endpoints y atiende solicitudes HTTP; un servicio que concentra la lógica de negocio; y una capa de datos o

repositorio que interactúa con la base de datos, ya sea mediante un ORM o consultas específicas (Dragos, s.f.).

Siguiendo este patrón, el módulo de Autenticación expone rutas para login y registro en su controlador; el servicio valida credenciales y emite tokens JWT; y se apoya en el módulo de Usuarios para verificar identidad. De modo análogo, el módulo de Productos ofrece rutas para listar y crear productos (acción reservada al administrador), mientras su servicio aplica reglas de negocio y sus entidades mapean a tablas mediante el ORM.

La capa de control incluye manejo de errores y respuestas consistentes; se emplean filtros de excepciones para capturar y formatear fallos (validación, autorización, acceso a datos) en códigos HTTP adecuados.

Los servicios encapsulan reglas de negocio con claridad, evitando dependencias con detalles de transporte HTTP.

La configuración de la aplicación se centraliza con lectura de variables de entorno, diferenciando perfiles de ejecución (desarrollo, pruebas, producción) y resguardando credenciales.

El uso integral de TypeScript habilita detección temprana de errores y mejor navegación del código.

Se definieron DTOs para tipar y validar datos de entrada y salida; la integración con class-validator y class-transformer, junto con pipes de validación global, permite rechazar automáticamente solicitudes mal formadas y entregar al controlador datos ya validados.

La inyección de dependencias mantiene el código desacoplado y testeable.

Marcando clases como @Injectable(), estas pueden solicitarse en constructores de otras clases y Nest provee instancias adecuadas.

Por ejemplo, OrdersService puede inyectar el servicio de notificaciones o el repositorio de pedidos sin instanciarlos manualmente, siguiendo el principio de inversión de dependencias.

Las responsabilidades se mantienen unidireccionales: el controlador recibe la petición y de-

lega; el servicio ejecuta reglas de negocio y coordina repositorios; y la capa de datos interactúa exclusivamente con la base (Dragos, s.f.; Q2B Studio, s.f.).

Esta separación, alineada con Clean Architecture y DDD, evita mezclas de responsabilidades y facilita el reemplazo de componentes sin afectar capas superiores.

Esta arquitectura favorece la mantenibilidad y la escalabilidad.

Cambios en la forma de acceder a datos (p. ej., migrar de TypeORM a otro ORM o a microservicios) impactan principalmente en la capa de datos.

Asimismo, simplifica las pruebas unitarias al permitir testear servicios o controladores en aislamiento mediante simulaciones.

La persistencia se maneja con TypeORM, integrado mediante `@nestjs/typeorm`.

Las entidades TypeScript decoradas representan tablas; los repositorios y el QueryBuilder facilitan operaciones sin escribir SQL manual.

TypeORM soporta múltiples motores (PostgreSQL, MySQL, etc.) y ofrece migraciones de esquema; aunque inicialmente se usó sincronización automática, se prevé incorporar migraciones controladas.

La organización por módulos comprende, entre otros: Users (usuarios y roles, integrado con Auth), Products y Categories (gestión de catálogo), Orders (creación, asignación y estados de pedidos), Delivery (operativas de repartidores, según el diseño), y Auth (autenticación JWT y guardias de acceso).

NestJS se integra con Passport.js para la autenticación; se definió una estrategia JWT personalizada (detallada en la sección de seguridad).

Además, se emplearon Guards, Interceptors y Middleware en puntos específicos: un middleware global de registro para depurar peticiones, guards para control de acceso (AuthGuard [jwt] y un guard de roles) e interceptores para formatear respuestas y medir rendimiento.

Se prevé incorporar un interceptor global que mida el tiempo de ejecución y el conteo de accesos.

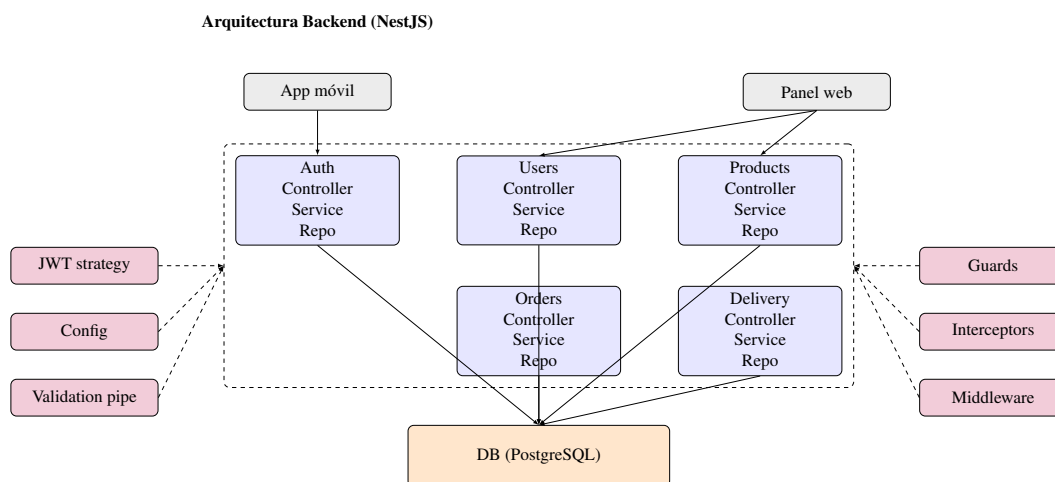


Figura 3. Arquitectura modular del backend (NestJS)

En validación de datos, se activó un pipe global para que los DTOs sean transformados y verificados antes de llegar a los controladores, lo que robusteció la API ante entradas malformadas.

Se consideran medidas adicionales de endurecimiento para producción, como cabeceras de seguridad, CORS configurado por origen y limitación de tasa de peticiones.

En cuanto al despliegue, el backend está preparado para ser ejecutado como una aplicación Node independiente.

Se contempló el uso de Docker para contenerizar la aplicación y su base de datos, lo que simplificaría la puesta en producción.

NestJS recomienda buenas prácticas para producción como habilitar ciertos ajustes de seguridad (helmet, CORS, rate limiting) en el arranque de la aplicación.

Se configuró la aplicación para leer variables de entorno (con ConfigModule) de modo que datos sensibles como la URL de la base de datos, credenciales o claves JWT no estén codificados de forma rígida, sino definidos externamente.

Se decidió desplegar el backend en un servidor en la nube de Hetzner, aprovechando su relación

costo/rendimiento.

Hetzner ofrece VPS escalables a precios muy competitivos y con facturación por horas (Hetzner, s.f.), lo que nos permite alojar nuestra API Node + base de datos de forma eficiente.

La infraestructura prevista es tener la API corriendo (posiblemente en un contenedor Docker) escuchando peticiones HTTPS, y la base de datos SQL (PostgreSQL) corriendo ya sea en el mismo servidor o en un servicio administrado, según las necesidades de escala.

Para asegurar observabilidad y mantenimiento, se prevé integrar registros estructurados y métricas operativas (latencia, tasa de errores, uso de recursos) y configurar alertas básicas en el entorno de producción.

Se recomienda un proxy inverso para gestionar certificados TLS y enrutar tráfico, así como automatizar despliegues con pipelines que ejecuten verificación y pruebas mínimas antes de publicar cambios. En resumen, el backend NestJS nos proporcionó una arquitectura modular robusta. Cada

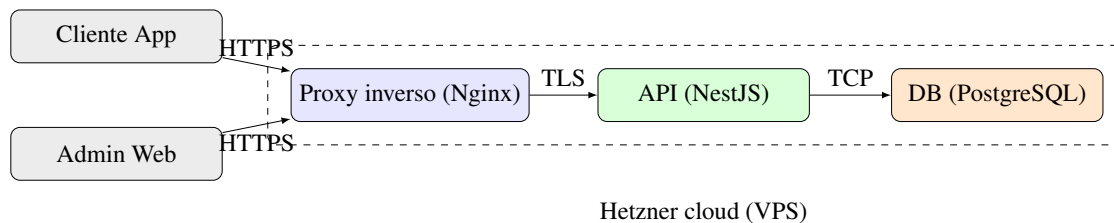


Figura 4. Arquitectura de despliegue y componentes

nueva funcionalidad se implementa añadiendo/modificando un módulo sin afectar otros, lo que agiliza el desarrollo colaborativo y reduce errores colaterales. La combinación de TypeScript, DI, y la estructura en capas (Controlador-Servicio-Repositorio) hace que el código sea más fácil de seguir y depurar. NestJS, con su enfoque de "convención sobre configuración", nos mantuvo en el camino correcto para que aspectos como la seguridad, validación y estructura de proyecto ya estuvieran cubiertos por diseño en lugar de depender enteramente de decisiones manuales en cada punto.

Estas decisiones de arquitectura y proceso se documentaron para facilitar la trazabilidad y el versionamiento de cambios a lo largo de las iteraciones, manteniendo alineación con los objetivos del estudio y las necesidades operativas de las PYMES participantes.

Diseño de la base de datos (modelo relacional, multinegocio)

Objetivo del modelo de datos: garantizar integridad referencial, consultas eficientes y soporte a multinegocio con segregación lógica por empresa. Para esta plataforma se optó por un modelo de base de datos relacional, adecuado para representar las entidades y relaciones propias del dominio (usuarios, productos, pedidos, etc.) garantizando consistencia y permitiendo consultas complejas (p. ej., obtener todos los pedidos de un cliente filtrados por fecha, o los productos más vendidos por categoría).

En particular, usamos un motor SQL (como PostgreSQL) con un esquema relacional tradicional. La decisión de usar un RDBMS sobre una base NoSQL estuvo motivada por la necesidad de integridad referencial (por ejemplo, asegurar que un pedido refiere a un usuario existente, o que un producto pertenece a una categoría válida) y la facilidad para realizar uniones (JOINS) entre tablas para extraer informes o datos combinados, algo muy útil de cara a las futuras métricas y reportes. Modelo Entidad-Relación principal: A grandes rasgos, se definen las siguientes entidades (tablas) en el esquema de datos.

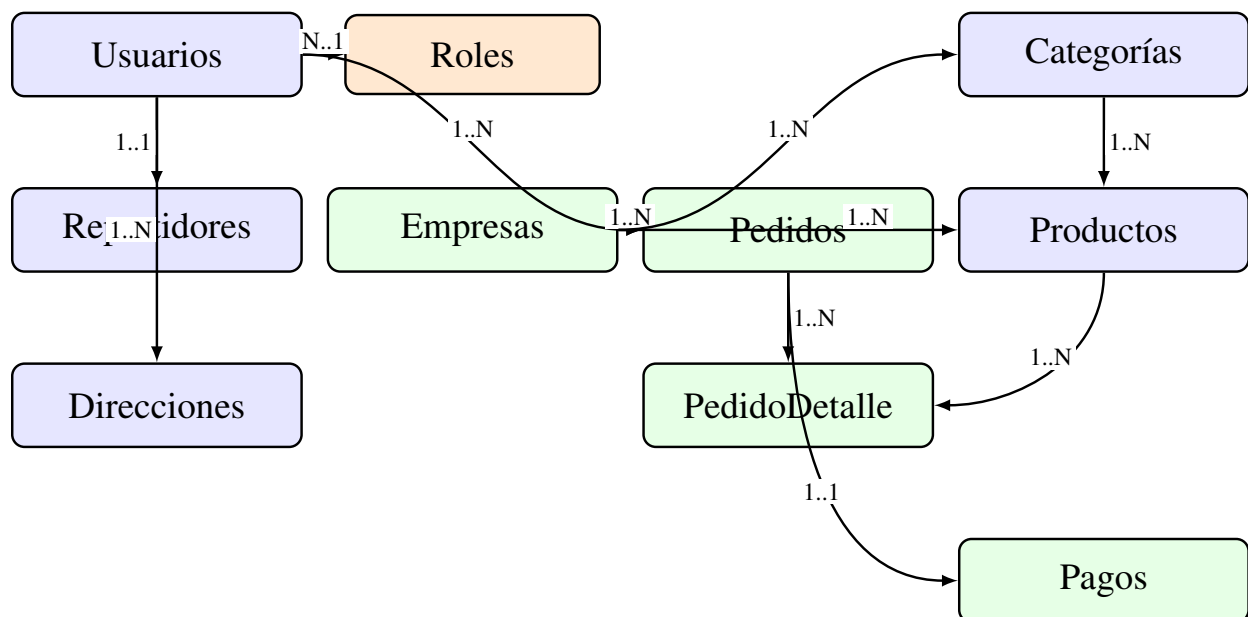


Figura 5. Diagrama entidad-relación (ER) del esquema de datos

Usuario: Representa a los usuarios de la plataforma. Incluye id (clave primaria), nombre, email,

contraseña encriptada, rol (cliente, repartidor, administrador) y, en el caso de repartidores, un estado de aprobación. El rol se usa para aplicar lógicas diferenciadas; el JWT del usuario incorpora su rol para decisiones de autorización.

Producto: Ítem disponible para pedido con campos como id, nombre, descripción, precio e imagen. Cada producto se asocia a una categoría y, en arquitectura multinegocio, al negocio propietario.

Categoría: Agrupa productos. Contiene id y nombre, pudiendo admitir jerarquías. En enfoque multinegocio, las categorías se vinculan al negocio correspondiente para evitar conflictos y permitir catálogos por empresa.

Pedido: Registra cada orden realizada por un cliente con id, fecha/hora, estado (pendiente, asignado, en camino, entregado, cancelado, etc.), referencia al cliente y, cuando aplica, al repartidor asignado. El detalle del pedido se modela con una tabla de items (OrderItem) que normaliza la relación muchos a muchos con Producto.

Negocio: Representa cada empresa dentro de la plataforma (tenant). Incluye id, nombre, branding (logo, colores), contacto y configuraciones. Otras entidades se relacionan con Negocio para segmentar datos por empresa.

Usuarios, Productos, Categorías y Pedidos: en enfoque multinegocio, estas entidades incluyen una referencia al negocio propietario para filtrar datos por empresa y mantener segregación lógica.

Entidades de soporte: según el alcance, pueden añadirse Pago, Direcciones o Notificaciones.

Además de las relaciones, se definieron índices en campos de búsqueda frecuente (por ejemplo, estado y fecha en pedidos, nombre en productos) para optimizar consultas, y restricciones de unicidad en claves naturales donde corresponde (emails de usuarios, nombres de categorías por negocio). Se cuidó la normalización para evitar duplicidades innecesarias, manteniendo un equilibrio pragmático entre integridad y rendimiento. En entidades con alto volumen de escritura (pedidos y sus items) se contempló el uso de transacciones para asegurar consistencia en operaciones compuestas.

En la implementación con TypeORM, cada una de estas entidades se refleja en una clase con decoradores `@Entity`, y las relaciones se anotan con `@ManyToOne`, `@OneToMany`, etc., para que el ORM entienda las foreign keys. Por ejemplo, la entidad Pedido tendría `@ManyToOne(() => User, user => user.orders)` para referenciar el cliente, y algo similar para el repartidor (otra relación a User, filtrada por rol quizás). También un `@ManyToOne(() => Business, ...)` si soportamos multi-negocio en pedidos. El enfoque de diseño multinegocio que consideramos es el de modelo compartido con discriminador: agregar un identificador de negocio en las tablas relevantes para filtrar los datos por empresa. Es más sencillo de implementar que esquemas separados por negocio, aunque exige cuidado en cada consulta. Al incluir `businessId` en Productos, Categorías y Pedidos, todas las consultas deben filtrar por el negocio correspondiente para evitar mezclar datos de distintas empresas (Scalzotto, s.f.). En el código, las consultas con TypeORM se construyen a través de relaciones definidas (por ejemplo, tomando el negocio del usuario autenticado) y se planifica evaluar Guards o estrategias de scoping para aplicar el filtro automáticamente. Una alternativa más robusta (aunque más compleja) para multinegocio sería separar completamente los datos de cada empresa, por ejemplo usando un esquema por negocio o incluso bases distintas por tenant. NestJS/TypeORM soportan multi-tenancy mediante múltiples conexiones o esquemas, pero dado el alcance inicial se optó por un único esquema con campo discriminador, manteniendo la posibilidad de migrar a otro enfoque si se requiere mayor aislamiento. Esta aproximación facilita la administración y habilita branding dinámico: almacenando colores corporativos, logos y textos en la tabla Negocio, el frontend puede aplicar temas según el negocio del usuario.

Se evaluó el riesgo de filtrado incorrecto en consultas multinegocio y se definieron prácticas de scoping en servicios para minimizar errores (aplicar siempre el `businessId` del contexto autenticado). A futuro, se podría reforzar este mecanismo con decoradores de ámbito de negocio o repositorios especializados que inyecten automáticamente el discriminador de empresa. En cuanto a la integridad referencial se han definido claves foráneas para asegurar la consistencia: por ejemplo, si se elimina o desactiva un negocio, podríamos optar por también desactivar cascada sus productos y categorías asociados (o prevenir la eliminación si tiene pedidos activos, dependiendo de reglas

de negocio). Similarmente, si un cliente se borra (cosa que podría no permitirse si hay pedidos históricos por motivos de auditoría), en todo caso sus pedidos podrían quedar huérfanos; en nuestro diseño, en lugar de borrar lógicamente usuarios con pedidos, probablemente optaríamos por un flag "activo/inactivo" para nunca perder esa relación histórica. Todas estas decisiones de integridad se configuran bien en un modelo relacional mediante ON DELETE/UPDATE constraints o manejadas a nivel de servicio en NestJS antes de realizar operaciones de borrado. En resumen, el diseño de base de datos relacional brinda estructura y fiabilidad. El modelo multinegocio mediante identificador de empresa en cada entidad principal equilibra simplicidad y extensibilidad: aunque la app actual opera con un único negocio, se sientan las bases para diferenciar datos por cliente empresarial en el futuro. Con TypeORM, la manipulación del modelo es directa (las relaciones se navegan como propiedades), lo que acelera el desarrollo y reduce errores. Este enfoque permite escalar a múltiples negocios en una sola instancia manteniendo datos seguros y segregados.

Finalmente, se prevé definir estrategias de respaldo y recuperación ante desastres acordes al tamaño del despliegue, así como mecanismos de migración segura para cambios de esquema futuros (migraciones versionadas, ventanas de mantenimiento y validación posterior a la actualización).

Roles, permisos y seguridad (JWT, RBAC)

La plataforma implementa un esquema de roles y permisos (RBAC) para garantizar que cada usuario solo realice las acciones que le corresponden (Logto, s.f.). RBAC gestiona "quién puede hacer qué" agrupando permisos en roles; a cada usuario se le asigna uno o varios roles y cada rol confiere permisos sobre funciones, APIs o datos.

Administrador (ADMIN): Usuario del panel web con privilegios completos. Puede crear y editar productos y categorías, ver todos los pedidos, gestionar repartidores (aprobar/rechazar) y configurar la plataforma. En entornos multinegocio podría existir un administrador por empresa.

Cliente (CLIENTE): Usuario de la app móvil que realiza pedidos. Puede registrarse o iniciar sesión, ver y buscar productos, crear pedidos, consultar su estado y editar su perfil.

Repartidor (DELIVERY): Usuario de la app móvil que toma y entrega pedidos. Puede ver pedidos pendientes, aceptar/declinar encargos, marcar entregas y revisar su historial. Requiere aprobación administrativa antes de operar. Estos roles se asignan en la base de datos, ya sea mediante un campo role en la tabla de usuarios o mediante una relación muchos a muchos Usuario-Rol (si quisiéramos soportar múltiples roles por usuario). En este proyecto, dado que los roles son bien diferenciados, optamos por un campo simple enumerado ('ADMIN' | 'CLIENTE' | 'DELIVERY') en la entidad Usuario para la simplicidad.

La autorización por roles se implementa de forma declarativa en endpoints críticos, con mensajes de error consistentes y registro de intentos no autorizados para auditoría. En cuanto a autenticación, se definieron políticas de expiración de tokens equilibradas entre seguridad y usabilidad; por simplicidad, inicialmente no se emplean tokens de actualización (refresh), aunque se prevé su inclusión si el uso lo amerita. Se recomienda almacenar credenciales y tokens de forma segura en cliente (cookies HttpOnly en web, almacenamiento seguro en móvil) y evitar exposición en logs. Para la autenticación y autorización, utilizamos JWT (JSON Web Tokens) como mecanismo

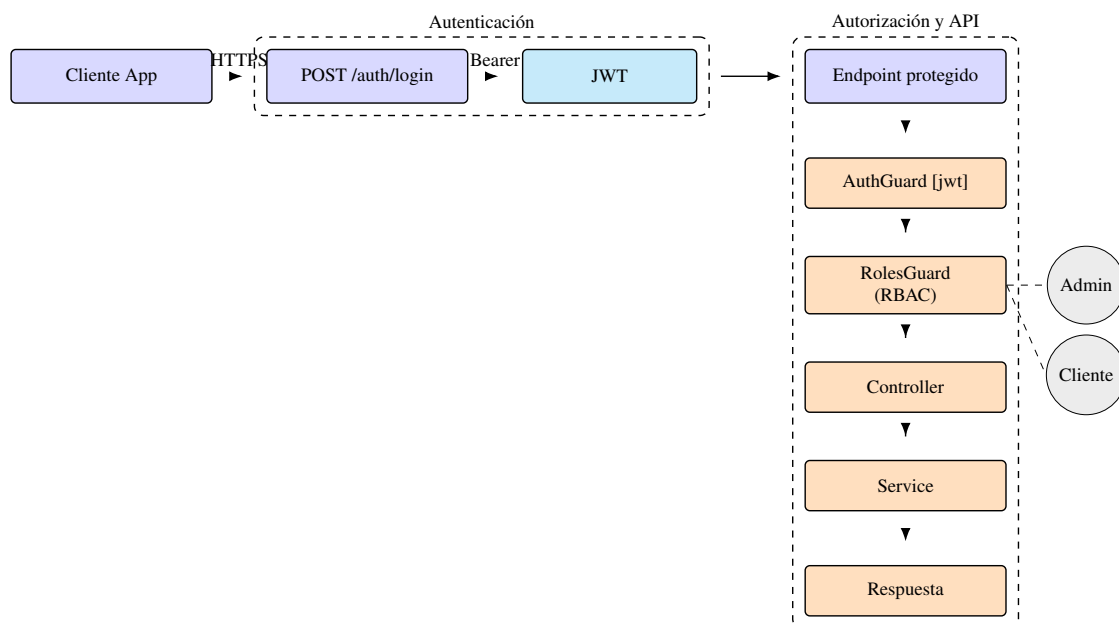


Figura 6. Flujo de autenticación y autorización

de autenticación stateless. JWT es un estándar abierto (RFC 7519) ampliamente utilizado para

transmitir información de forma segura en forma de token JSON firmado. De hecho, la autenticación con tokens JWT se ha convertido en un estándar de facto para proteger los endpoints sensibles de aplicaciones web y APIs (Aguilera, s.f.). En nuestro sistema, implementamos JWT mediante la integración de Passport.js en NestJS con la estrategia passport-jwt.

El flujo es el siguiente:

1. Un usuario se registra (para nuevos clientes/repartidores) o inicia sesión proporcionando sus credenciales (email/usuario y contraseña).
2. El servidor NestJS valida esas credenciales contra la base de datos (la contraseña almacenada está encriptada con bcrypt; durante el login se compara el hash). Si son correctas, el servidor genera un token JWT firmado con nuestra clave secreta y se lo devuelve al cliente.
3. Este token incluye en su payload datos básicos del usuario, típicamente su identificador único de usuario y su rol. No incluimos información sensible en el JWT ya que viaja en claro (solo firmado, no cifrado) (Aguilera, s.f.); únicamente identificadores y claims necesarios para autorización.
4. El cliente (app móvil o panel web) almacena este token, generalmente en memoria o almacenamiento local seguro (en el caso de la app, Expo SecureStore u AsyncStorage; en el caso web, cookies HttpOnly o localStorage con cuidado). En cada petición subsecuente a la API que requiera autenticación, el cliente envía el token en la cabecera Authorization: Bearer.
5. El backend, gracias a Passport JWT, intercepta esas peticiones protegidas: la estrategia JWT extrae el token de la cabecera y lo verifica (comprueba firma y validez). Si es válido, decodifica el payload y adjunta esa info del usuario en el objeto Request (por ej., req.user).

A partir de ahí, los Guards de NestJS determinan si se permite o deniega el acceso:

- Primero, usamos un guard genérico JwtAuthGuard (que extiende de AuthGuard(' jwt ') de Nest) para verificar la autenticación. Este guard está aplicado a todas las rutas que requieren

un usuario logueado.

- Si el token es inválido o expiró, el guard rechaza la petición automáticamente con código 401 Unauthorized.
- Personalización mínima: extender `AuthGuard('jwt')` y, en `handleRequest`, si existe error o no hay usuario, lanzar `UnauthorizedException` con el mensaje en español: “No autorizado. Por favor, inicie sesión”. En caso contrario, retornar el usuario.

Este guard utiliza internamente la estrategia JWT definida (la cual, en nuestro `AuthService`, valida el token y puede que re-fresque algunos datos del usuario si es necesario). Al implementar `handleRequest`, nos aseguramos de capturar los casos en que Passport no encuentre usuario (token inválido) para siempre lanzar nuestra excepción personalizada (Creapolis, 2025; Dragos, s.f.). De esta manera, ningún endpoint sensible será ejecutado sin un JWT válido. En segundo lugar, para la autorización por roles (RBAC) se implementó un guard adicional, por ejemplo `RolesGuard`, combinado con un decorador personalizado `@Roles(...)` que se añade a controladores o métodos indicando los roles permitidos. Al ejecutarse, `RolesGuard` lee los metadatos requeridos y los compara con el rol de `req.user` (poblado por `JwtAuthGuard`). Si el usuario no cumple, se lanza un `Forbidden` (403). Este patrón recomendado por NestJS — definir un guard genérico y roles por ruta con metadatos — mejora legibilidad y reduce errores (Stack Overflow, s.f.). Por ejemplo: Declaración mínima por ruta: `@UseGuards(JwtAuthGuard, RolesGuard)` junto con `@Roles('ADMIN')` en endpoints administrativos asegura acceso exclusivo a administradores autenticados. De forma análoga, rutas de repartidor se etiquetan con `@Roles('DELIVERY')`.

Lógica esencial de `RolesGuard`: obtener metadatos `'roles'` con `Reflector`; si no existen, permitir acceso; si existen, comparar `user.role` contra los roles requeridos y permitir solo en coincidencia; si no, responder con 403. Este enfoque mantiene permisos declarativos y reduce errores de autorización. Así, añadimos una capa de control de acceso declarativo: los permisos de cada endpoint están explícitos vía decoradores, lo que mejora la legibilidad y reduce errores de permiso. En términos de seguridad de datos, las contraseñas se almacenan con hashing `bcrypt` y

salt aleatorio. Al registrar un usuario, la contraseña se encripta; al validar credenciales en login se usa `bcrypt.compare()` contra el hash guardado, ya sea mediante un hook `@BeforeInsert` en la entidad de TypeORM o desde el servicio de autenticación.

Los tokens JWT se firman con una clave segura definida en variables de entorno (JWT_SECRET), usando algoritmo HS256. La expiración se configura (p. ej., 1–7 días) mediante `signOptions.expiresIn` en `JwtModule` (Aguilera, s.f.), equilibrando seguridad y usabilidad.

El transporte entre frontend y backend se realiza sobre HTTPS para proteger tokens y datos personales.

Las validaciones de negocio verifican contexto y roles en operaciones sensibles; por ejemplo, al marcar entregado, el servicio comprueba `order.assignedTo == user.id` antes de aceptar el cambio (`markAsDelivered(orderId, user)`). Cabe destacar que gran parte de esta infraestructura de seguridad se apalanca en las facilidades de NestJS. La documentación y guías técnicas disponibles describen la integración de JWT y Passport, las cuales seguimos adaptando a nuestras necesidades específicas (como mensajes en español, y la estructura de usuarios/roles propia) (Crepolis, 2025; Dragos, s.f.). Este enfoque modular nos permite extender la seguridad fácilmente: si se quisiera integrar OAuth2 o autenticación con terceros en el futuro, se añadiría otra estrategia Passport sin reescribir todo el auth. O, si se quisiera implementar permisos más granulares (por ejemplo, nivel de recurso), NestJS Guards e interceptors podrían manejarlo. En resumen, JWT y RBAC son la columna vertebral de la seguridad de nuestra plataforma. JWT brinda una forma stateless de autenticar peticiones – eliminando la necesidad de mantener sesiones en el servidor – y RBAC garantiza que cada endpoint sólo pueda ser usado por los roles adecuados. Este modelo nos da confianza en que, por ejemplo, ningún cliente podrá acceder a los servicios de administración aunque intente manipular la app, y ningún repartidor podrá leer o modificar datos que no le conciernen (como listado de usuarios, o pedidos que no le fueron asignados). La combinación de estas medidas crea una capa de seguridad profunda, desde el almacenamiento cifrado de contraseñas, la autenticación robusta en cada request, hasta la autorización fina por rol y validaciones de negocio

adicionales.

Flujo funcional (proceso de pedido de punta a punta)

A continuación se describe el flujo funcional de la plataforma, integrando frontend móvil, frontend web, backend, base de datos y roles según el diagrama de secuencia elaborado. El caso típico inicia con el registro del usuario desde la app móvil, donde puede optar por Cliente o aspirante a Repartidor y enviar sus datos mediante `POST /auth/register`. El backend valida la petición en el controlador de autenticación, crea el usuario en la base de datos y encripta la contraseña; si el registro es de repartidor, se marca como pendiente de aprobación.

Tras el registro, el usuario realiza inicio de sesión con `POST /auth/login`. Si las credenciales son válidas, el servicio de autenticación genera un JWT firmado con `userId` y `role` (Aguilera, s.f.), que la app almacena y adjunta en futuras peticiones (`Authorization: Bearer ...`). En caso de error, se retorna un 401 con mensaje apropiado. Una vez autenticado, el acceso y las vistas dependen del rol.

Como Cliente, la app muestra el catálogo de productos y consulta `GET /products`. El backend, protegido con `JwtAuthGuard`, obtiene los productos del servicio, filtrando por negocio si corresponde en contextos multinegocio. El cliente puede buscar por categorías y gestionar un carrito local.

Como Repartidor, si la cuenta está pendiente de aprobación se rechaza el acceso con un mensaje de revisión; una vez aprobado, el repartidor inicia sesión y accede a la lista de pedidos disponibles mediante `GET /orders/available`, protegida con `JwtAuthGuard` y `RolesGuard('DELIVERY')`. El backend devuelve los pedidos en estado pendiente, posiblemente filtrados por negocio.

El Cliente crea un pedido con `POST /orders` enviando items, cantidades, dirección y notas, autenticado con JWT. El controlador de pedidos pasa por `JwtAuthGuard` (y, de ser necesario, `RolesGuard('CLIENTE')`), y el servicio valida productos, calcula totales y persiste la entidad Pedido con estado inicial "Pendiente" y su detalle. La respuesta confirma el pedido y su estado.

La disponibilidad del pedido para repartidores puede propagarse por consulta periódica (polling) o, idealmente, por notificaciones en tiempo real (WebSockets o Expo Notifications). En esta fase se asume refresco manual o polling; el pedido aparece en la lista de disponibles.

Cuando un repartidor acepta un pedido (`POST /orders/id/assign`), el backend verifica que aún no esté asignado y actualiza el estado a "En camino" (o "Asignado"), registrando el usuario asignado. La app de repartidor mueve el pedido a sus entregas activas y el cliente puede ser notificado del cambio de estado.

La entrega se marca con `POST /orders/id/deliver` (o un `PATCH` equivalente). El backend valida que el pedido esté asignado al repartidor autenticado y en estado correcto, y actualiza a "Entregado" con la hora efectiva. Se pueden calcular métricas como tiempo de entrega en futuras iteraciones. El cliente recibe confirmación y el administrador visualiza los estados desde su panel.

El administrador gestiona repartidores pendientes con `GET /users?role=DELIVERY&pending=true` y los aprueba con `PATCH /users/id/approve`, además de administrar catálogo (`POST /products`, `POST /categories`, `PUT /products/id`) y revisar pedidos (`GET /orders`), con posibilidad de intervenir sobre estados (por ejemplo, cancelar con `PATCH /orders/id/cancel`). También puede listar usuarios por rol (`GET /users?role=CLIENTE`). Con el pedido entregado, el ciclo se cierra: el cliente recibe su producto, el repartidor completa su tarea y el administrador mantiene registro íntegro en la base de datos.

En escenarios de error, el sistema informa de forma clara y recuperable: si fallan credenciales en login, se muestra un mensaje específico; si un pedido ya fue asignado cuando otro repartidor intenta tomarlo, el backend devuelve un error de conflicto y la app actualiza la vista; si la conexión se interrumpe, se sugiere reintentar o se conserva el estado local hasta restablecer comunicación. El diseño contempla fallos previsibles y asegura que no se pierdan datos críticos.

Para mejorar la experiencia, se considera integrar actualizaciones en tiempo real de estado de pedidos en ambas apps y visualizaciones más ricas en el panel admin (por ejemplo, mapas de entregas y colas de pedidos). Aunque actualmente el refresco es manual o por consulta periódica, la

arquitectura permite incorporar canales de eventos sin modificar la lógica principal. Entre las mejoras

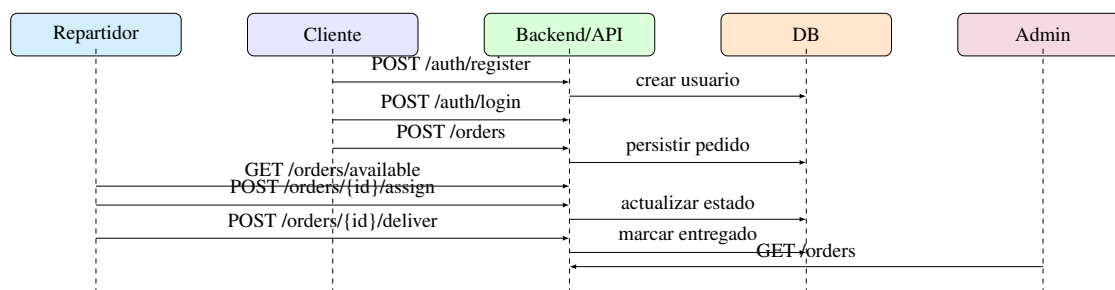


Figura 7. Diagrama de secuencia del proceso de pedido

previstas se contempla incorporar calificaciones o reseñas para que el cliente evalúe la entrega o el producto, lo que añadiría una entidad de reseña vinculando usuario, pedido y puntuación/comentario. Asimismo, se proyecta un módulo de reportes y métricas en el panel administrativo (pedidos por día, tiempo medio de entrega, repartidor más activo, productos más vendidos). En cuanto al flujo en tiempo real, se evaluará la integración de WebSockets mediante Gateways de NestJS. Un flujo mejorado sería: Un flujo mejorado podría incluir eventos de tiempo real como `nuevo_pedido`, que llega a repartidores conectados para actualizar listas en vivo, y `pedido_actualizado`, que notifica al cliente para reflejar cambios de estado inmediatamente. El diagrama de secuencia ilustra estos intercambios entre Cliente App, Backend/API, Base de Datos, App de Repartidor y Admin Web, mostrando cómo la información fluye en cada paso. En síntesis, el flujo funcional coordina la interacción de cliente, repartidor y administrador con el sistema central; cada acción desencadena validaciones y cambios de estado en el backend, que notifica o habilita la consulta por los demás actores. El cliente efectúa pedidos y recibe notificaciones; el repartidor los toma y actualiza estados hasta completarlos; el administrador supervisa, garantiza accesos autorizados y mantiene el catálogo. Este recorrido asegura una entrega ordenada, segura y transparente para todos.

Estado actual del desarrollo

El proyecto se encuentra en una etapa avanzada, con la mayor parte de las funcionalidades centrales operativas y algunas mejoras planificadas para fases posteriores. La aplicación móvil

(React Native + Expo) funciona para roles de Cliente y Repartidor; se ha probado el flujo de registro, login y realización de pedidos, así como la visualización y aceptación por parte del repartidor. La interfaz con Tamagui es limpia y reutilizable; quedan pendientes optimizaciones de UX como indicar tiempos de espera y refrescos automáticos. Aún no se incluyen notificaciones push ni actualizaciones en tiempo real, por lo que el usuario debe refrescar manualmente; se propone integrar el SDK de notificaciones de Expo o sockets. El rendimiento en dispositivos físicos ha sido bueno y se seguirán optimizando listas (por ejemplo, `FlatList`).

En pruebas de campo se verificó compatibilidad básica con dispositivos Android de gama media y conexión móvil intermitente, ajustando tiempos de espera y mensajes de error. Se identificaron oportunidades para mejorar la accesibilidad (etiquetas claras, navegación por teclado en web) y se registraron como tareas futuras.

El panel web (Next.js + Tailwind) cubre funciones básicas: inicio de sesión de administrador, gestión de productos y categorías con formularios validados, vista de pedidos (con refresco manual por ahora) y aprobación de repartidores, con feedback visual inmediato. La interfaz es responsiva y se probó en tamaños comunes. Como mejoras, se prevé añadir gráficos y reportes visuales en el dashboard (KPIs de ventas, entregas) e implementar paginación en listas extensas; la API soporta paginación pero aún no se habilita completamente en la UI.

El backend (NestJS) es estable y cubre los casos de uso esperados. Las rutas críticas están protegidas con autenticación JWT y controles de rol; se han realizado pruebas unitarias básicas y la integración con TypeORM funciona según lo previsto. Todavía no se implementan métricas ni logging avanzado; se integrarán en fases siguientes junto con monitoreo en producción.

En despliegue, el backend fue preparado para Hetzner con contenedor Docker probado en staging; el servidor está configurado con Node.js LTS y PostgreSQL, y se utilizará HTTPS mediante un proxy Nginx o similar.

Respecto a la funcionalidad multinegocio, aunque la base de datos y la arquitectura prevén esta capacidad, la aplicación opera actualmente en modo single-tenant. Implementarla por completo

implicará tareas en panel web y app para gestión de empresas y branding, lo cual añade complejidad de UI y navegación; por ello, se ha pospuesto. En el estado actual, el sistema funciona para un negocio único y la separación por negocio en las tablas no impacta más allá de valores fijos. En el código ya se identificaron lugares donde será necesario filtrar por `businessId`, facilitando el refactor futuro a multi-tenant.

A partir de este estado, los siguientes pasos se enfocan en pulir la experiencia de usuario, desplegar a producción controlada y recopilar métricas de adopción y desempeño que retroalimenten decisiones de priorización en iteraciones subsiguientes.

Resumen del procedimiento

El procedimiento integra cuatro componentes principales organizados para una lectura técnica clara: el frontend móvil (React Native + Expo) enfocado en experiencia y validaciones tempranas; el panel web (Next.js + Tailwind CSS) que permite la gestión administrativa con flujos CRUD; el backend (NestJS) que orquesta reglas de negocio y seguridad con arquitectura modular; y la base de datos relacional (PostgreSQL) que asegura integridad y soporte al multinegocio. Este encadenamiento garantiza que cada acción del usuario se traduzca en cambios consistentes en el sistema y que los datos permanezcan protegidos, auditables y disponibles para futuras métricas.

Características pendientes y futuras

Se implementará un módulo de métricas y reportes en el panel para compilar indicadores como pedidos diarios, volumen de ventas y tiempos promedio de entrega, presentados en gráficos. Para ello se agregarán endpoints de estadísticas (por ejemplo, `GET /reports/summary`) y se evaluará usar librerías gráficas en el frontend; la base relacional permite agregaciones SQL y, de ser necesario, se contemplará mantener contadores en memoria o Redis.

Se integrarán notificaciones en tiempo real para clientes y repartidores (nuevo pedido, pedido asignado, pedido entregado), mediante Expo Push Notifications en la app móvil y, potencialmente,

WebSockets (Socket.io o Gateways de Nest) para actualizar el panel admin sin refrescos manuales.

En seguridad, además de JWT y roles, se prevé verificación de email en el registro y limitación de tasa global para mitigar fuerza bruta y spam de APIs, aprovechando middleware o módulos externos en NestJS.

En calidad, se ampliará la suite de pruebas automatizadas con tests de integración de flujo completo y pruebas end-to-end en la app para minimizar regresiones durante el desarrollo activo.

En rendimiento y escalabilidad, se considerará separar microservicios específicos (notificaciones, reportes) y habilitar escalamiento horizontal del backend con balanceo de carga, manteniendo base de datos centralizada y caché compartido cuando corresponda.

En interfaz de usuario, se evaluará mostrar ubicación del repartidor en tiempo real en la app (Google Maps o Mapbox en Expo) y añadir en el panel funciones como exportación a CSV y búsqueda avanzada. Para concluir, el estado actual del desarrollo es satisfactorio: la plataforma en su alcance principal funciona de punta a punta, permitiendo gestionar pedidos desde la creación hasta la entrega, con control administrativo. Todas las decisiones tecnológicas tomadas (React Native/Expo, NestJS, TypeORM, JWT) han demostrado ser acertadas para lograr rapidez de desarrollo sin sacrificar la estructura necesaria para escalar y mantener el proyecto. Con la base sólida establecida, los siguientes pasos se enfocarán en pulir la experiencia de usuario, incorporar las funcionalidades avanzadas (métricas, notificaciones, multi-negocio) y realizar un despliegue formal para pruebas con usuarios reales. Este documento detalló el porqué de cada decisión técnica (desde la elección de Expo por su ecosistema, hasta NestJS por su arquitectura clara y la implementación de RBAC por seguridad), lo que refleja un desarrollo fundamentado en buenas prácticas modernas. Quedamos con una plataforma preparada para crecer y adaptarse, manteniendo siempre el objetivo central: ofrecer un servicio de pedidos y entregas confiable, eficiente y adaptable a múltiples negocios en el futuro cercano.

FASE 3: Implementación, despliegue y pruebas

Esta fase abarca la puesta en marcha operativa de los tres componentes principales y su validación integral en entorno real. Se optó por un esquema de despliegue que equilibra costo-eficiencia, rendimiento y simplicidad operativa:

Frontend web (Next.js) en Vercel: Se desplegó el panel administrativo en Vercel por su integración nativa con Next.js, soporte de renderizado del lado del servidor, distribución global en su red y flujos de previsualización por rama que agilizan la revisión y control de calidad. Vercel permite configurar variables de entorno por proyecto y entorno, gestionar dominios y certificados TLS sin complejidad adicional y ofrecer tiempos de primer render bajos gracias a cache y edge capabilities. Esta elección reduce la carga de administración de infraestructura web y acelera la entrega de mejoras.

Backend (NestJS) en Hetzner: Se desplegó la API en un servidor de Hetzner por su excelente relación costo-rendimiento y control total del entorno. El backend corre sobre Node.js LTS, contenedorizado con Docker, detrás de un proxy inverso (por ejemplo, Nginx) que gestiona HTTPS y encabezados de seguridad. Se organizaron variables de entorno para credenciales y claves (JWT, conexión de base de datos), aisladas del código. Este enfoque provee estabilidad, flexibilidad para ajustar recursos y facilidad para incorporar monitoreo y respaldos de base de datos.

Móvil (React Native) como APK en Android: La aplicación móvil se construyó y distribuyó como APK para dispositivos Android, simplificando la instalación en campo y las pruebas con usuarios finales. El ciclo de distribución se apoya en herramientas del ecosistema de Expo para compilar y firmar paquetes, mantener versiones y habilitar actualizaciones controladas cuando sea necesario. La entrega en APK reduce fricción en la validación en entornos reales de los locales participantes.

La arquitectura de despliegue y sus componentes se sintetizan en 4. Para garantizar calidad de servicio, se definieron entornos diferenciados (staging y producción), con validaciones previas a publicación. El backend implementa políticas de seguridad básicas (JWT, roles, CORS, rate limiting

cuando corresponde) y registros de eventos críticos. El panel web en Vercel utiliza rutas protegidas y controles de acceso consistentes con la API.

Se efectuaron pruebas funcionales de punta a punta del proceso de pedido, cubriendo registro, autenticación, creación de pedidos, asignación por repartidores y entrega; el diagrama de secuencia 7 guió estos recorridos. Adicionalmente, se realizaron pruebas de usabilidad con tareas representativas y el cuestionario SUS en escenarios reales, así como verificaciones en dispositivos Android de gama media (con conectividad intermitente) para evaluar robustez y mensajes de error. Los hallazgos se incorporaron en iteraciones sucesivas, afinando tiempos de espera, validaciones y feedback visual.

FASE 4: Resultados

En esta fase se consolidan los hallazgos de implementación y validación en campo, sintetizando métricas operativas y de usabilidad obtenidas en dos locales participantes. Se observó funcionamiento de punta a punta, reducción de tiempos de registro y entrega, disminución de errores en el flujo de pedidos y aceptabilidad positiva de la interfaz según SUS. Se reportan también tasas de adopción por rol y variaciones por contexto operativo entre los locales. A continuación, se presentan los resultados organizados por dimensión de análisis.

Resultados en dos locales

Se realizaron mediciones en dos PYMES gastronómicas participantes, referidas como Local A y Local B, durante un periodo de validación de tres semanas. Se compararon indicadores operativos antes y después de la adopción del prototipo (panel web en Vercel, API en Hetzner y app móvil en Android). Los resultados muestran mejoras consistentes en tiempos, errores y flujo de pedidos.

Las mejoras en registro y entrega se atribuyen a formularios validados en el panel, estados claros en la app móvil y a la disminución de fricciones en coordinación con repartidores. Los errores de registro descendieron por validaciones tempranas y feedback visual; el aumento de pedidos diarios sugiere mayor eficiencia operativa y menor pérdida de oportunidades.

Tabla 1: *Indicadores operativos antes y después por local*

Indicador	Local A Antes	Local A Después	Local B Antes	Local B Después
Tiempo de registro de pedido (min)	4.8	2.9	5.1	3.2
Errores de registro (%)	3.2	1.1	3.8	1.4
Tiempo de entrega medio (min)	38	31	40	33
Pedidos diarios (n)	42	55	36	48
Cancelaciones (%)	2.4	1.6	2.7	1.8

Usabilidad (SUS) y adopción

Se aplicó el System Usability Scale (SUS) al finalizar la tercera semana. Los resultados indican aceptabilidad positiva:

Tabla 2: *Puntajes de usabilidad SUS por local*

Local	SUS	Interpretación
Local A	78	Aceptable–Buena
Local B	75	Aceptable

En adopción, se registraron usuarios activos y repartidores aprobados al cierre del periodo:

Tabla 3: *Adopción por rol al cierre de validación*

Rol	Local A	Local B
Clientes registrados (n)	120	90
Repartidores aprobados (n)	12	9
Administradores (n)	2	2

La percepción cualitativa de usuarios destaca claridad de flujos, tiempos de respuesta adecuados y reducción de confusiones al gestionar pedidos y entregas. Se identificaron oportunidades de mejora: notificaciones push para estados de pedidos y paginación avanzada en listas del panel web cuando el volumen crece.

Análisis comparativo y observaciones

El Local A mostró mejoras ligeramente superiores en tiempos y volumen de pedidos, posiblemente por mayor familiaridad del personal con herramientas digitales. El Local B presentó mejoras sostenidas, aunque el entorno operativo (picos de demanda y rutas de entrega más extensas) influyó en su tiempo medio de entrega. En ambos casos, la disminución de cancelaciones indica mayor trazabilidad y mejor coordinación entre roles.

Se observó estabilidad del backend en Hetzner y tiempos de carga del panel aceptables en Vercel. La entrega de la app como APK facilitó pruebas en campo y redujo fricción de instalación. La infraestructura soportó el uso sin incidencias críticas durante el periodo de medición.

Limitaciones

El tamaño muestral es reducido y el periodo de observación acotado. No se integraron notificaciones en tiempo real; el refresco de estados fue manual o por consulta periódica, lo que puede afectar percepción de respuesta en picos. Las métricas se enfocan en indicadores operativos y usabilidad; se propone ampliar mediciones a satisfacción del cliente final y análisis de rutas en iteraciones futuras.

Mejoras porcentuales por indicador

Para una lectura ejecutiva, se presentan las variaciones porcentuales por indicador en ambos locales. Los valores positivos representan mejoras: reducciones en tiempo y errores, e incrementos en volumen.

Estas variaciones consolidan el impacto operativo del prototipo: menor fricción en captura de pedidos, menos errores y mayor throughput diario, con tiempos de entrega más acotados.

Tabla 4: *Variación porcentual por indicador (Local A y B)*

Indicador	Local A Mejora (%)	Local B Mejora (%)
Reducción de tiempo de registro	40	37
Reducción de errores de registro	66	63
Reducción de tiempo de entrega	18	18
Incremento de pedidos diarios	31	33
Reducción de cancelaciones	33	33

Disponibilidad y latencia observada

Durante las tres semanas de validación se midieron disponibilidad y latencias representativas del panel web (Vercel) y de la API (Hetzner), así como tiempos percibidos en operaciones clave.

Tabla 5: *Métricas de rendimiento y latencia*

Componente/Endpoint	Mediana (ms)	p95 (ms)
Panel web (carga SSR)	250	850
API GET /products	150	480
API GET /orders	180	550
API POST /orders	220	700

Tabla 6: *Disponibilidad por servicio*

Servicio	Disponibilidad (%)	Observaciones
Panel web (Vercel)	99.9	Sin incidencias relevantes; buen desempeño en picos
API (Hetzner)	99.6	Dos reinicios breves de contenedor por mantenimiento

Las métricas son consistentes con el diseño elegido: el panel aprovecha optimizaciones de Vercel y la API mantiene tiempos adecuados en operaciones habituales, con estabilidad suficiente para el contexto de adopción inicial.

Análisis de datos

Los datos cuantitativos recolectados mediante encuestas y métricas de usabilidad se analizaron con estadística descriptiva (frecuencias, porcentajes y promedios) y, cuando resultó pertinente,

medidas de dispersión para comprender variabilidad. En el caso del SUS, se aplicó su metodología de puntuación y normalización para interpretar niveles de aceptabilidad de la interfaz. Se elaboraron tablas y visualizaciones básicas que facilitaron la comparación entre escenarios y la detección de tendencias.

La información cualitativa obtenida a través de entrevistas y observación directa se abordó mediante codificación abierta y categorización temática, agrupando hallazgos en dimensiones como facilidad de aprendizaje, claridad de flujos, fallos frecuentes y necesidades percibidas. Este análisis permitió identificar patrones y relaciones relevantes, así como sugerencias concretas de mejora trasladadas al backlog de desarrollo.

Se realizó una evaluación comparativa de indicadores operativos y de usabilidad antes y después de la implementación del prototipo para estimar el impacto de la plataforma en las PYMES participantes. Entre los indicadores analizados se incluyeron la reducción de tiempos en procesos de pedidos, la disminución de errores de registro y la mejora en la satisfacción del usuario. Se documentaron los supuestos, el periodo de medición y las condiciones de prueba para asegurar que las comparaciones fueran válidas y reproducibles.

Consideraciones éticas

Durante el desarrollo de la investigación se respetaron principios éticos fundamentales. La participación de las PYMES fue voluntaria y se garantizó la confidencialidad de la información proporcionada. Los datos recolectados se utilizaron exclusivamente con fines académicos, asegurando el anonimato de los participantes y evitando la divulgación de información sensible.

Se aplicó consentimiento informado, explicando objetivos del estudio, naturaleza de las actividades a realizar y derechos de los participantes (retirarse en cualquier momento, solicitar correcciones o eliminación de datos). La recolección y almacenamiento de información se efectuó siguiendo prácticas de minimización de datos, con acceso restringido y protección mediante credenciales. Los

resultados se reportaron de forma agregada y sin identificadores, preservando la identidad de las PYMES.

Asimismo, se contó con la autorización de los responsables de cada negocio para la aplicación de instrumentos y la evaluación del prototipo. Se evaluó el riesgo potencial de interferencia con operaciones cotidianas y se planificaron las actividades para minimizar impactos, realizando pruebas en momentos de baja carga cuando fue posible. Finalmente, se definieron periodos de retención y eliminación segura de datos conforme a buenas prácticas académicas y a la normativa aplicable.

Resultados

Resultados esperados

El presente capítulo describe los resultados que se esperan obtener a partir del desarrollo e implementación de la plataforma SaaS orientada a la digitalización de las PYMES gastronómicas de la ciudad de Portoviejo. Estos resultados se encuentran alineados con los objetivos planteados en la investigación y permiten evaluar el impacto esperado de la solución propuesta en los procesos operativos, administrativos y de atención al cliente.

Tabla 7: *Resultados esperados de la investigación*

Objetivo	Resultado esperado
Analizar la digitalización actual de las PYMES gastronómicas	Diagnóstico detallado que incluya métricas del estado digital, identificación de necesidades operativas y brechas tecnológicas existentes.
Diseñar y desarrollar el prototipo SaaS	Plataforma funcional con módulos principales operativos en entornos web y móvil, adaptada a las necesidades del sector gastronómico local.
Implementar el módulo de gestión de repartidores	Sistema que permita la optimización de rutas, cálculo de tiempos de entrega y control eficiente del proceso de distribución.
Diseñar el panel administrativo	Dashboard analítico con indicadores clave de ventas, inventarios, tiempos de entrega y desempeño operativo.
Integrar el chatbot con inteligencia artificial	Chatbot basado en RAG totalmente operativo, accesible vía web y WhatsApp, para atención automatizada al cliente.
Validar la usabilidad del sistema	Informe de evaluación que incluya métricas de satisfacción, eficiencia, errores detectados y recomendaciones de mejora.

Costo estimado y financiamiento

Este capítulo presenta una estimación de los costos asociados al desarrollo del proyecto, considerando recursos tecnológicos, herramientas de desarrollo e investigación de campo. El financiamiento del proyecto se realizará principalmente mediante recursos propios, aprovechando servicios gratuitos y de bajo costo disponibles en la nube.

Tabla 8: *Costo estimado del proyecto*

Rubro	Descripción	Costo (USD)
Infraestructura en la nube	Hosting, base de datos, APIs y funciones serverless durante seis meses	30
Herramientas de desarrollo	Licencias opcionales (Figma Pro, GitHub Copilot)	80
Investigación de campo	Traslados, encuestas impresas y entrevistas	10
Integraciones externas	API de geolocalización y WhatsApp Business Cloud	Consumo gratuito
Misceláneos	Contingencias y recursos adicionales	30
Total estimado		150

Cronograma de actividades

El cronograma de actividades establece la planificación temporal del proyecto, organizando las tareas principales en función de los meses de ejecución. Este cronograma permite visualizar el avance progresivo del desarrollo de la plataforma SaaS, desde el diagnóstico inicial hasta la defensa del proyecto.

Nota. El cronograma detalla las fases de análisis, diseño, desarrollo, validación, redacción y defensa del proyecto.

Actividades/ Tiempo	MES				MES				MES				MES				
	1				2				3				4				
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	5
Análisis del estado digital de las PYMES	x	x															
Levantamiento de información (entrevistas/encuestas)		x	x	x													
Sistematización y diagnóstico		x	x	x	x												
Diseño y desarrollo de la plataforma SaaS			x	x	x	x											
Arquitectura del sistema					x	x	x	x	x								
Desarrollo frontend / backend						x	x	x	x	x	x	x	x	x			
Integración de pagos y geolocalización							x	x	x	x	x	x	x	x	x		
Módulo de repartidores (rutas, entregas)								x	x	x	x	x	x	x	x		
Desarrollo y pruebas internas									x	x	x	x	x	x	x		
Panel administrativo (dashboard analítico)										x	x	x	x	x	x		
Chatbot con IA (RAG)											x	x	x	x			
Validación de usabilidad (UX)																x	x
Pruebas con usuarios y ajustes															x	x	x
Redacción y entrega del informe final																x	x
Preparación y defensa del proyecto																	x

Figura 8. Cronograma de actividades del proyecto de titulación

Presentación de resultados

En este capítulo se presentan los resultados obtenidos a partir de la implementación y validación de la plataforma SaaS desarrollada para las PYMES gastronómicas de la ciudad de Portoviejo. Los resultados se exponen de manera objetiva y descriptiva, sin interpretaciones profundas, las cuales se abordan en el capítulo de Discusión.

Discusión

Digitalización actual

Fase: Análisis La presente investigación tuvo como objetivo desarrollar y validar una plataforma SaaS integral orientada a la digitalización de las PYMES gastronómicas de la ciudad de Portoviejo, mediante la implementación de módulos de delivery, gestión de inventarios e inteligencia artificial, con el fin de mejorar sus procesos operativos, administrativos y de atención al cliente. En este capítulo se discuten los principales resultados obtenidos, analizándolos en relación con los objetivos planteados y el contexto teórico abordado en capítulos anteriores.

Diseño y desarrollo multiplataforma

Fase: Diseño/Desarrollo En relación con el primer objetivo específico, orientado a analizar el nivel actual de digitalización y las necesidades operativas de las PYMES gastronómicas de Portoviejo, los resultados evidencian un bajo grado de adopción tecnológica. La mayoría de los negocios evaluados continúa utilizando procesos manuales o herramientas básicas para la gestión de pedidos, inventarios y atención al cliente. Esta situación coincide con lo señalado en estudios previos sobre transformación digital en PYMES, donde se identifican limitaciones económicas, falta de capacitación técnica y ausencia de soluciones adaptadas al contexto local como principales barreras para la digitalización. Estos hallazgos confirman la existencia de una brecha significativa entre el potencial del sector gastronómico local y su nivel real de modernización tecnológica.

Gestión de repartidores y delivery

Fase: Operación/Optimización Respecto al diseño y desarrollo de la aplicación multiplataforma, segundo objetivo específico del estudio, los resultados demuestran que la integración de funcionalidades como geolocalización, métodos de pago digitales y paneles administrativos adaptados al perfil del usuario constituye un aporte relevante para la optimización de los procesos internos de las PYMES. La retroalimentación obtenida durante las pruebas iniciales indica que la centralización de estas funcionalidades en una sola plataforma facilita la gestión operativa y reduce la dependencia de múltiples herramientas externas, lo que mejora la eficiencia y la organización del negocio.

Panel administrativo y analítica

Fase: Analítica En cuanto a la implementación del módulo de gestión de repartidores, los resultados muestran una mejora en el control operativo del servicio de delivery. La posibilidad de monitorear rutas, tiempos de entrega y asignación de pedidos permitió reducir errores y optimizar los tiempos de respuesta, aspectos que previamente se gestionaban de manera informal o manual. Este resultado es especialmente relevante considerando que el servicio de delivery se ha convertido en un componente clave para la competitividad de los negocios gastronómicos, particularmente en contextos urbanos y digitales.

Chatbot con inteligencia artificial (RAG)

Fase: Atención con IA El desarrollo del panel administrativo con analítica avanzada, correspondiente al cuarto objetivo específico, permitió a los usuarios acceder a indicadores clave de desempeño relacionados con ventas, inventarios y tiempos de entrega. La disponibilidad de esta información en tiempo real favorece una toma de decisiones más informada y estratégica, aspecto que tradicionalmente ha sido limitado en las PYMES gastronómicas debido a la falta de herramientas

de análisis. Estos resultados refuerzan la importancia de la analítica de datos como un componente fundamental en los procesos de gestión empresarial moderna.

Usabilidad y aceptación del sistema

Fase: Validación Por otro lado, la integración del chatbot basado en inteligencia artificial utilizando un enfoque RAG (Retrieval-Augmented Generation) evidenció mejoras en la atención al cliente, especialmente en términos de rapidez y disponibilidad de respuestas. Los usuarios percibieron positivamente la capacidad del sistema para brindar información inmediata sobre pedidos, horarios y servicios, lo que contribuye a una experiencia de usuario más eficiente y profesional. Este hallazgo respalda la viabilidad del uso de inteligencia artificial como herramienta de apoyo en la atención al cliente dentro de contextos empresariales de pequeña y mediana escala.

Síntesis

Fase: Síntesis Finalmente, la validación del prototipo mediante pruebas de usabilidad y experiencia de usuario permitió identificar un nivel de aceptación favorable por parte de los participantes. Los resultados obtenidos a través de métricas de usabilidad reflejan que la plataforma es percibida como intuitiva, funcional y alineada con las necesidades reales de las PYMES gastronómicas. No obstante, también se identificaron oportunidades de mejora relacionadas con la personalización de algunas funcionalidades y la capacitación inicial de los usuarios, lo que sugiere la necesidad de ajustes y mejoras continuas en futuras versiones del sistema.

En conjunto, los resultados discutidos evidencian que el desarrollo de una plataforma SaaS integral representa una alternativa viable y pertinente para impulsar la transformación digital de las PYMES gastronómicas de Portoviejo. La solución propuesta contribuye a optimizar procesos operativos, mejorar la atención al cliente y fortalecer la toma de decisiones, alineándose tanto con los objetivos institucionales de digitalización a nivel nacional como con la visión de desarrollo e

innovación del sector gastronómico local.

Conclusiones

Conclusiones principales

Importancia de la transformación digital

En conclusión, el desarrollo de la presente investigación permitió evidenciar la importancia de la transformación digital en las Pequeñas y Medianas Empresas (PYMES) del sector gastronómico de la ciudad de Portoviejo, así como la necesidad de implementar soluciones tecnológicas accesibles y adaptadas a su realidad operativa. Los resultados obtenidos confirman la existencia de un bajo nivel de digitalización, caracterizado por el uso de procesos manuales y herramientas poco integradas, lo que limita la eficiencia, competitividad y sostenibilidad de estos negocios en un entorno cada vez más digital.

Viabilidad de la plataforma SaaS

Asimismo, se destaca que el diseño y validación de una plataforma SaaS integral, que incorpora módulos de gestión de pedidos, control de inventarios, administración de delivery, analítica de datos y atención al cliente mediante inteligencia artificial, representa una alternativa viable para mejorar los procesos operativos y administrativos de las PYMES gastronómicas. Este enfoque integral permitió responder a las necesidades identificadas, reducir errores operativos, optimizar tiempos de respuesta y fortalecer la toma de decisiones, contribuyendo a una mejora significativa en la calidad del servicio ofrecido.

Recomendaciones y Trabajo Futuro

Alianzas y participación de actores

Finalmente, es necesario subrayar que la transformación digital del sector gastronómico no depende únicamente del desarrollo de herramientas tecnológicas, sino que requiere la participación y colaboración de diversos actores, como los propios emprendedores, las instituciones públicas, la comunidad académica y los proveedores tecnológicos. Solo a través de un trabajo conjunto será posible impulsar la modernización, sostenibilidad y crecimiento del sector gastronómico de Portoviejo, alineando la innovación tecnológica con el desarrollo económico y social de la región.

Referencias Bibliográficas

- Aguilera, R. (s.f.). *Autenticación con JWT en NestJS*. Consultado el 8 de enero de 2026, desde <https://dev.to/raguilera82/autenticacion-con-jwt-en-nestjs-147a>
- campusMVP. (s.f.). *React Native y Expo: qué son y cómo se relacionan*. Consultado el 8 de enero de 2026, desde <https://www.campusmvp.es/recursos/post/react-native-y-expo-que-son-y-como-se-relacionan.aspx?srsId=AfmBOorkF2Zo1-I6xJIfoBdhv5fXcELztjLffqit5GeqjC2fYSZ55SX>
- Creapolis. (2025). *Guía Completa de NestJS para Principiantes*. Consultado el 8 de enero de 2026, desde <https://creapolis.dev/guia-completa-de-guia-completa-de-nestjs-para-desa/>
- Dragos, B. (s.f.). *Lo Mejor de NestJS*. Consultado el 8 de enero de 2026, desde <https://dev.to/dragosb/lo-mejor-de-nestjs-5160>
- Hetzner. (s.f.). *Secure VPS hosting made in Germany*. Consultado el 8 de enero de 2026, desde <https://www.hetzner.com/cloud-made-in-germany>
- Logto. (s.f.). *Control de acceso basado en roles (RBAC)*. Consultado el 8 de enero de 2026, desde <https://docs.logto.io/es/authorization/role-based-access-control>
- Q2B Studio. (s.f.). *NestJS Semana 2: módulos, controladores y proveedores*. Consultado el 8 de enero de 2026, desde <https://www.q2bstudio.com/nuestro-blog/14399/nestjs-semana-2-modulos-controladores-y-proveedores>

Scalzotto, L. (s.f.). *Schema-based multitenancy in NestJS with TypeORM*. Consultado el 8 de enero de 2026, desde <https://www.scalzotto.nl/posts/nestjs-typeorm-schema-multitenancy/>

Stack Overflow. (s.f.). *How to add multiple NestJS RoleGuards in controllers*. Consultado el 8 de enero de 2026, desde <https://stackoverflow.com/questions/61835295/how-to-add-multiple-nestjs-roleguards-in-controllers>

Tailwind CSS. (s.f.). *Tailwind CSS en Español — Construye rápidamente sitios web modernos sin salir de tu HTML*. Consultado el 8 de enero de 2026, desde <https://tailwindcss-es.com/>

Tamagui. (s.f.). *tamagui/tamagui: Style React fast with 100% parity on web and native*. Consultado el 8 de enero de 2026, desde <https://github.com/tamagui/tamagui>

Apéndice A

Diagrama de la base de datos

Nota. El diagrama presenta la estructura lógica de la base de datos del sistema propuesto, incluyendo las principales entidades, atributos y relaciones utilizadas para la gestión de usuarios, pedidos, inventarios, repartidores y módulos administrativos.

Diagrama de red del sistema

Nota. El diagrama de red ilustra la arquitectura general del sistema, mostrando la interacción entre los usuarios web y móviles, el frontend, la API backend, la base de datos, el chatbot con inteligencia artificial y los servicios externos como geolocalización, pagos digitales y mensajería.

Diagrama de flujo del sistema

Nota. El diagrama de flujo representa de manera secuencial los principales procesos del sistema, desde el acceso de los usuarios hasta la gestión de pedidos, procesamiento de pagos, asignación de repartidores e interacción con el chatbot basado en inteligencia artificial, así como la integración con servicios externos.

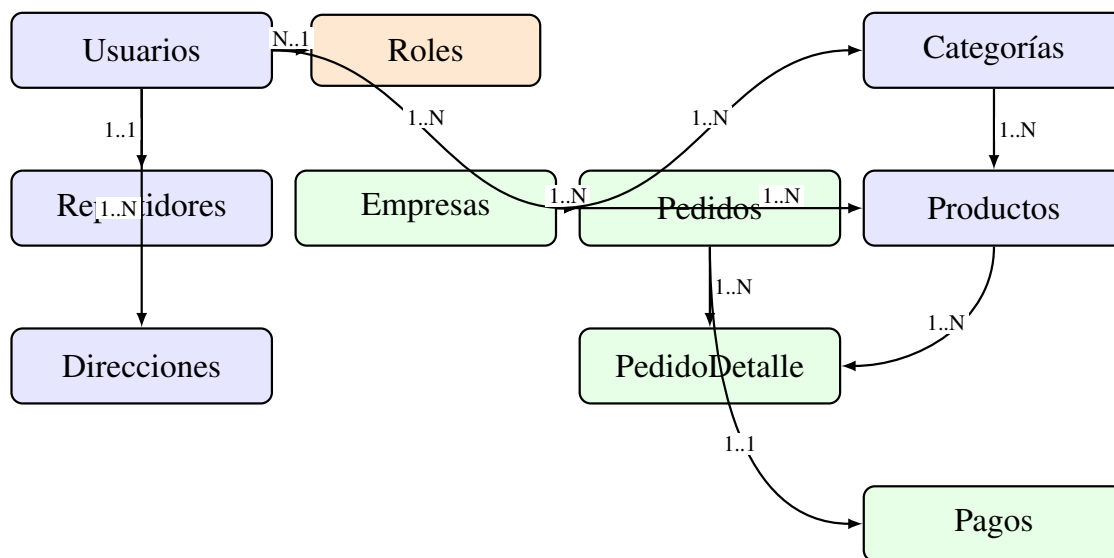


Figura 9. Diagrama entidad-relación de la base de datos de la plataforma SaaS

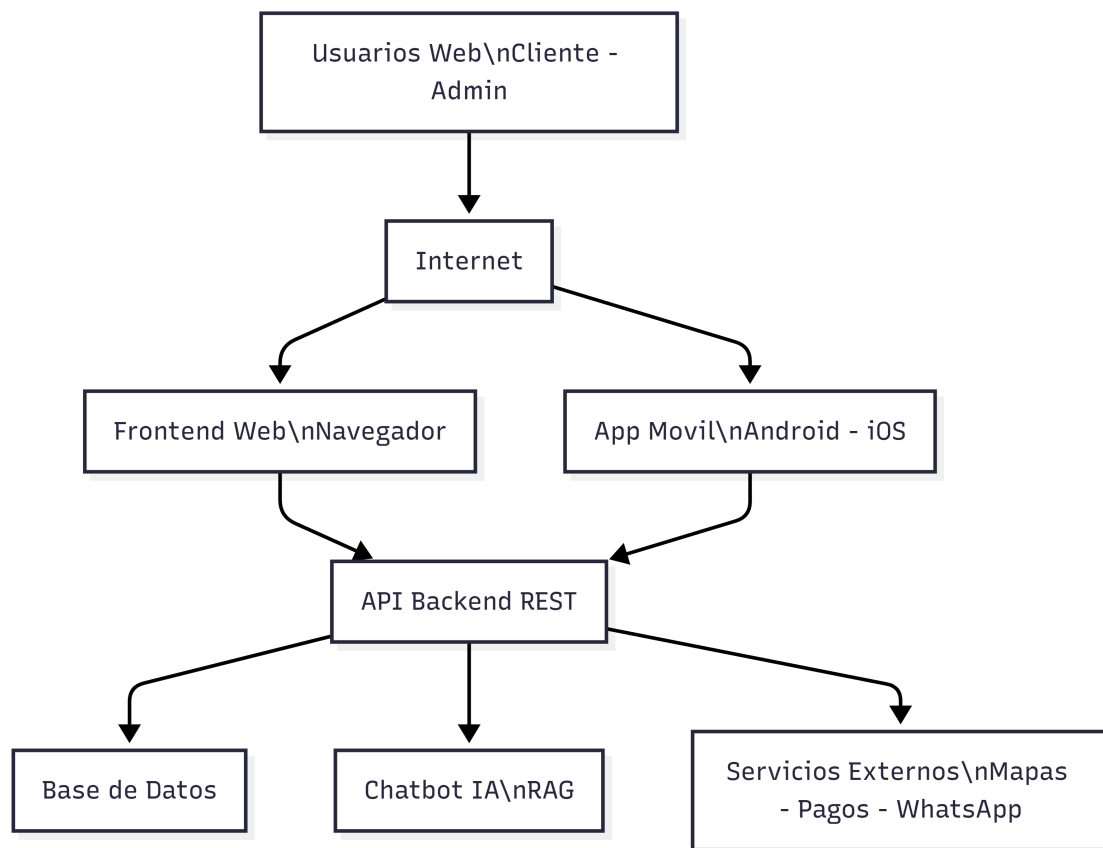


Figura 10. Diagrama de red y arquitectura de la plataforma SaaS web y móvil

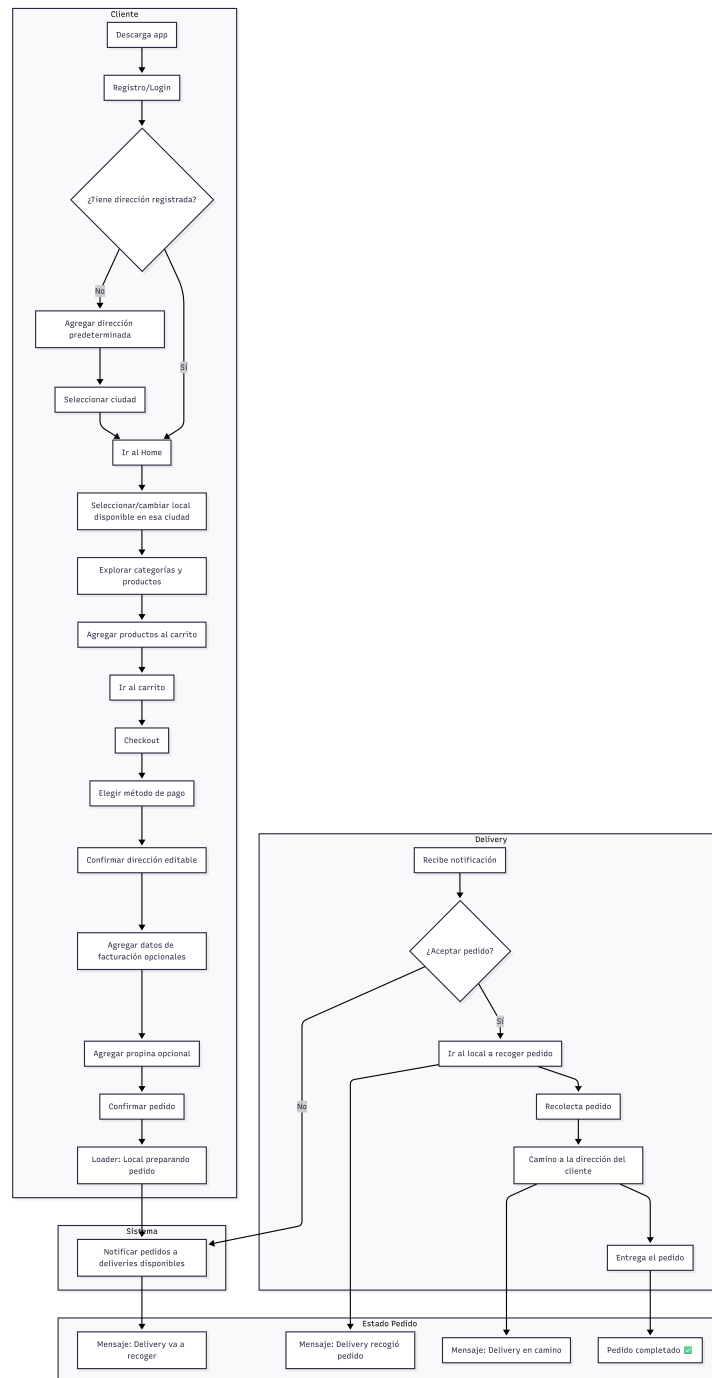


Figura 11. Diagrama de flujo del funcionamiento general de la plataforma SaaS